Embedded Linux system development

BeagleBone Black variant

Practical Labs

bootlin

https://bootlin.com

July 17, 2026

About this document

Updates to this document can be found on https://bootlin.com/training/embedded-linux.

This document was generated from LaTeX sources found on https://github.com/bootlin/training-materials.

More details about our training sessions can be found on https://bootlin.com/training.

Copying this document

© 2004-2026, Bootlin, https://bootlin.com.



This document is released under the terms of the Creative Commons CC BY-SA 3.0 license. This means that you are free to download, distribute and even modify it, under certain conditions.

Corrections, suggestions, contributions and translations are welcome!

Training setup

Download files and directories used in practical labs

Install lab data

For the different labs in this course, your instructor has prepared a set of data (kernel images, kernel configurations, root filesystems and more). Download and extract its tarball from a terminal:

```
$ cd
$ wget https://bootlin.com/doc/training/embedded-linux/embedded-linux-bbb-labs.tar.xz
$ tar xvf embedded-linux-bbb-labs.tar.xz
```

Lab data are now available in an `embedded-linux-bbb-labs` directory in your home directory. This directory contains directories and files used in the various practical labs. It will also be used as working space, in particular to keep generated files separate when needed.

Update your distribution

To avoid any issue installing packages during the practical labs, you should apply the latest updates to the packages in your distro:

```
$ sudo apt update
$ sudo apt dist-upgrade
```

You are now ready to start the real practical labs!

Install extra packages

Feel free to install other packages you may need for your development environment. In particular, we recommend to install your favorite text editor and configure it to your taste. The favorite text editors of embedded Linux developers are of course *Vim* and *Emacs*, but there are also plenty of other possibilities, such as Visual Studio Code[1], *GEdit*, *Qt Creator*, *CodeBlocks*, *Geany*, etc.

It is worth mentioning that by default, Ubuntu comes with a very limited version of the `vi` editor. So if you would like to use `vi`, we recommend to use the more featureful version by installing the `vim` package.

More guidelines

Can be useful throughout any of the labs

- Read instructions and tips carefully. Lots of people make mistakes or waste time because they missed an explanation or a guideline.
- Always read error messages carefully, in particular the first one which is issued. Some people stumble on very simple errors just because they specified a wrong file path and didn't pay enough attention to the corresponding error message.
- Never stay stuck with a strange problem more than 5 minutes. Show your problem to your colleagues or to the instructor.
- You should only use the `root` user for operations that require super-user privileges, such as: mounting a file system, loading a kernel module, changing file ownership, configuring the network. Most regular tasks (such as downloading, extracting sources, compiling...) can be done as a regular user.

[1] This tool from Microsoft is Open Source! To try it on Ubuntu: `sudo snap install code --classic`

© 2004-2026 Bootlin, CC BY-SA license

- If you ran commands from a root shell by mistake, your regular user may no longer be able to handle the corresponding generated files. In this case, use the `chown -R` command to give the new files back to your regular user.
  Example: `$ sudo chown -R myuser.myuser linux/`

 © 2004-2026 Bootlin, CC BY-SA license

Building a cross-compiling toolchain

Objective: Learn how to compile your own cross-compiling toolchain for the musl C library

After this lab, you will be able to:

- Configure the *crosstool-ng* tool
- Execute *crosstool-ng* and build up your own cross-compiling toolchain

Setup

Go to the `$HOME/embedded-linux-bbb-labs/toolchain` directory.

For this lab, you need a system or VM with a least 4 GB of RAM.

Install needed packages

Install the packages needed for this lab:

```
$ sudo apt install build-essential git autoconf bison flex texinfo help2man gawk libtool-bin \
    libncurses5-dev unzip gettext python3 rsync
```

Getting Crosstool-ng

Let's download the sources of Crosstool-ng, through its git source repository, and switch to a commit that we have tested:

```
$ git clone https://github.com/crosstool-ng/crosstool-ng
$ cd crosstool-ng/
$ git checkout crosstool-ng-1.28.0
```

Building and installing Crosstool-ng

As we are not building Crosstool-ng from a release archive but from a git repository, we first need to generate a `configure` script and more generally all the generated files that are shipped in the source archive for a release:

```
$ ./bootstrap
```

We can then either install Crosstool-ng globally on the system, or keep it locally in its download directory. We'll choose the latter solution. As documented at https://crosstool-ng.github.io/docs/install/#hackers-way, do:

```
$ ./configure --enable-local
$ make
```

Then you can get Crosstool-ng help by running

```
$ ./ct-ng help
```

© 2004-2026 Bootlin, CC BY-SA license

Configure the toolchain to produce

A single installation of Crosstool-ng allows to produce as many toolchains as you want, for different architectures, with different C libraries and different versions of the various components.

Crosstool-ng comes with a set of ready-made configuration files for various typical setups: Crosstool-ng calls them *samples*. They can be listed by using `./ct-ng list-samples`.

We will load the Cortex A8 sample. Load it with the `./ct-ng` command.

Then, to refine the configuration, let's run the `menuconfig` interface:

```
$ ./ct-ng menuconfig
```

In `Path and misc options`:

- If not set yet, enable `Try features marked as EXPERIMENTAL`

In `Target options`:

- Set `Use specific FPU` (`ARCH_FPU`) to `vfpv3`.
- Set `Floating point` to `hardware (FPU)`.

In `Toolchain options`:

- Set `Tuple's vendor string` (`TARGET_VENDOR`) to `training`.
- Set `Tuple's alias` (`TARGET_ALIAS`) to `arm-linux`. This way, we will be able to use the compiler with `arm-linux-gcc`, a shorter name that the name based on complete toolchain tuple.

In `Operating System`:

- Set `Version of linux` to the closest, but older version to 6.6. It's important that the kernel headers used in the toolchain are not more recent than the kernel that will run on the board (v6.6).

In `C-library`:

- If not set yet, set `C library` to `musl` (`LIBC_MUSL`)

In `C compiler`:

- Set `Version of gcc` to `15.2.0`.
- Make sure that `C++` (`CC_LANG_CXX`) is enabled

In `Debug facilities`:

- Remove all options here. Some debugging tools can be provided in the toolchain, but they can also be built by filesystem building tools.

Explore the different other available options by traveling through the menus and looking at the help for some of the options. Don't hesitate to ask your trainer for details on the available options. However, remember that we tested the labs with the configuration described above. You might waste time with unexpected issues if you customize the toolchain configuration.

Produce the toolchain

Nothing is simpler:

```
$ ./ct-ng build
```

The toolchain will be installed by default in `$HOME/x-tools/`. That's something you could have changed in Crosstool-ng's configuration.

And wait!

 © 2004-2026 Bootlin, CC BY-SA license

Testing the toolchain

You can now test your toolchain by adding `$HOME/x-tools/arm-training-linux-musleabihf/bin/` to your `PATH` environment variable and compiling the simple `hello.c` program in your main lab directory with `arm-linux-gcc`:

```
$ arm-linux-gcc -o hello hello.c
```

You can use the `file` command on your binary to make sure it has correctly been compiled for the ARM architecture.

Did you know that you can still execute this binary from your x86 host? To do this, install the QEMU user emulator, which just emulates target instruction sets, not an entire system with devices:

```
$ sudo apt install qemu-user
```

Now, try to run QEMU ARM user emulator:

```
$ qemu-arm hello
qemu-arm: Could not open '/lib/ld-musl-armhf.so.1': No such file or directory
```

What's happening is that `qemu-arm` is missing the shared library loader (compiled for ARM) that this binary relies on. Let's find it in our newly compiled toolchain:

```
$ find ~/x-tools -name ld-musl-armhf.so.1
```

```
/home/tux/x-tools/arm-training-linux-musleabihf/arm-training-linux-musleabihf/sysroot/lib/
ld-musl-armhf.so.1
```

We can now use the `-L` option of `qemu-arm` to let it know where shared libraries are:

```
$ qemu-arm -L ~/x-tools/arm-training-linux-musleabihf/arm-training-linux-musleabihf/sysroot \
    hello
```

```
Hello world!
```

Cleaning up

Do this only if you have limited storage space. In case you made a mistake in the toolchain configuration, you may need to run Crosstool-ng again, keeping generated files would save a significant amount of time.

To save about 9 GB of storage space, do a `./ct-ng clean` in the Crosstool-NG source directory. This will remove the source code of the different toolchain components, as well as all the generated files that are now useless since the toolchain has been installed in `$HOME/x-tools`.

© 2004-2026 Bootlin, CC BY-SA license

Bootloader - U-Boot

Objectives: Set up serial communication, compile and install the U-Boot bootloader, use basic U-Boot commands, set up TFTP communication with the development workstation.

As the bootloader is the first piece of software executed by a hardware platform, the installation procedure of the bootloader is very specific to the hardware platform. There are usually two cases:

- The processor offers nothing to ease the installation of the bootloader, in which case the JTAG has to be used to initialize flash storage and write the bootloader code to flash. Detailed knowledge of the hardware is of course required to perform these operations.
- The processor offers a monitor, implemented in ROM, and through which access to the memories is made easier.

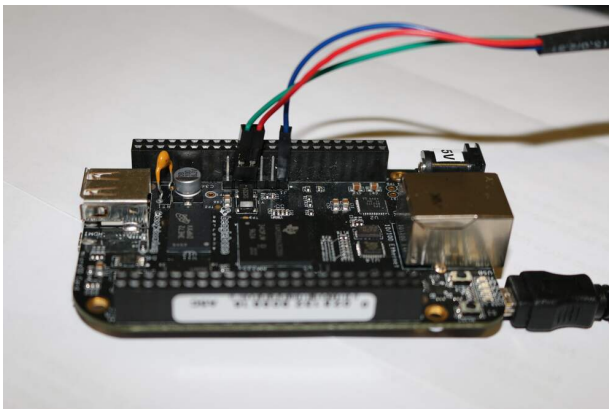
The AM3358 SoC on the BeagleBone falls into the second category. The monitor integrated in the ROM reads the SD card to search for a valid bootloader.

Go to the `$HOME/embedded-linux-bbb-labs/bootloader` directory.

Setting up serial communication with the board

The Beaglebone serial connector is exported on the 6 pins close to one of the 48 pins headers. Using your special USB to Serial adapter provided by your instructor, connect the ground wire (blue) to the pin closest to the power supply connector (let's call it pin 1), and the `TX` (red) and `RX` (green) wires to the pins 4 (board `RX`) and 5 (board `TX`)[2].

You always should make sure that you connect the `TX` pin of the cable to the `RX` pin of the board, and vice versa, whatever the board and cables that you use.



Once the USB to Serial connector is plugged in, a new serial port should appear: `/dev/ttyUSB0`.

You can also see this device appear by looking at the output of `sudo dmesg`.

To communicate with the board through the serial port, install a serial communication program, such as `picocom`:

[2] See https://www.olimex.com/Products/USB-Modules/Interfaces/USB-SERIAL-F for details about the USB to Serial adapter that we are using.

 © 2004-2026 Bootlin, CC BY-SA license

```
$ sudo apt install picocom
```

If you run `ls -l /dev/ttyUSB0`, you can also see that only `root` and users belonging to the `dialout` group have read and write access to the serial console. Therefore, you need to add your user to the `dialout` group:

```
$ sudo adduser $USER dialout
```

Important: for the group change to be effective, you have to reboot your computer (at least on Ubuntu 22.04) and log in again. A workaround is to run `newgrp dialout`, but it is not global. You have to run it in each terminal.

Run `picocom -b 115200 /dev/ttyUSB0`, to start serial communication on `/dev/ttyUSB0`, with a baudrate of 115200. If you wish to exit `picocom`, press `[Ctrl][a]` followed by `[Ctrl][x]`.

There should be nothing on the serial line so far, as the board is not powered up yet.

It is now time to power up your board by plugging in the mini-USB (BeagleBone Black case) or micro-USB (BeagleBone Black Wireless case) cable supplied by your instructor to your PC.

See what messages you get on the serial line. You should see U-boot start on the serial line, if there was a valid U-Boot and SPL on the board's eMMC.

Compiling U-Boot and SPL

Download U-Boot:

```
$ git clone https://source.denx.de/u-boot/u-boot.git/
$ cd u-boot
$ git checkout v2024.04
```

Get an understanding of U-Boot's configuration and compilation steps by reading the `README` file, and specifically the *Building the Software* section.

Basically, you need to:

1. Specify the cross-compiler prefix (the part before `gcc` in the cross-compiler executable name):

   ```
   $ export CROSS_COMPILE=arm-linux-
   ```

2. Run `$ ls configs/ | grep am335` to see all predefined configurations. The one that supports our board is not obvious: it's `am335x_evm_defconfig` and not `am335x_boneblack_vboot_defconfig` which is only for *verified boot* on BeagleBone Black.

3. So, run `$ make am335x_evm_defconfig`.

4. Now that you have a valid initial configuration, you can now run `$ make menuconfig` to further edit your bootloader features.

   Here, though, the default configuration works fine for our needs, so no change is necessary.

   Install the following packages which should be needed to compile U-Boot for your board:

   ```
   $ sudo apt install libssl-dev device-tree-compiler swig \
         python3-dev python3-setuptools
   ```

5. Finally, depending on your board, run

   ```
   make DEVICE_TREE=am335x-boneblack
   ```

   or

```
make DEVICE_TREE=am335x-boneblack-wireless
```

which will build U-Boot [3]. The `DEVICE_TREE` variable specifies the specific Device Tree that describes our hardware board. Alternatively, if you wish to run just `make`, specify our board's device tree name on `Device Tree Control` → `Default Device Tree for DT Control` option.

Preparing a bootable micro-SD card

The TI romcode will look for an `MLO` (*MMC Load*) file in a FAT partition on an SD card. This is precisely what U-Boot compiled for us, together with the U-Boot binary (`u-boot.img`).

Let's prepare an SD card with such a partition.

Plug the SD card your instructor gave you on your workstation. Type the `sudo dmesg` command to see which device is used by your workstation. In case the device is `/dev/mmcblk0`, you will see something like

```
[46939.425299] mmc0: new high speed SDHC card at address 0007
[46939.427947] mmcblk0: mmc0:0007 SD16G 14.5 GiB
```

The device file name may be different (such as `/dev/sdb` if the card reader is connected to a USB bus (either internally or using a USB card reader).

In the following instructions, we will assume that your SD card is seen as `/dev/mmcblk0` by your PC workstation.

Type the `mount` command to check your currently mounted partitions. If SD partitions are mounted, unmount them:

```
$ sudo umount /dev/mmcblk0p*
```

We will erase the existing partition table by simply zero-ing the first 16 MiB of the SD card:

```
$ sudo dd if=/dev/zero of=/dev/mmcblk0 bs=1M count=16
```

Now, let's use the `cfdisk` command to create the first partition that we need to boot the board: we are going to use:

```
$ sudo cfdisk /dev/mmcblk0
```

If `cfdisk` asks you to `Select a label type`, choose `dos`. This corresponds to traditional partitions tables that DOS/Windows would understand. `gpt` partition tables are needed for disks bigger than 2 TB.

In the `cfdisk` interface, delete existing partitions, then create only one primary partition, starting from the beginning, with the following properties:

- Size: 64MB big
- Type: `W95 FAT32 (LBA)` (`c` choice)
- Bootable flag enabled

Press `Write` when you are done.

We will create further partitions in a later lab, when we need them.

To make sure that partition definitions are reloaded on your workstation, remove the SD card and insert it again.

Now create a FAT32 filesystem on this new partition: [4]

[3] You can speed up the compiling by using the `-jX` option with `make`, where X is the number of parallel jobs used for compiling. Twice the number of CPU cores is a good value.

[4] Ubuntu uses version 4.2 of `mkfs.vfat` and the FAT generated by this version of the command is incompatible with what the TI AM335x romcode expects. Passing the `-a` option is a workaround as described on a Bootlin blog post.

 © 2004-2026 Bootlin, CC BY-SA license

```
sudo mkfs.vfat -a -F 32 -n boot /dev/mmcblk0p1
```

You can now make your workstation automatically mount this partition by removing the SD card and plugging it back. It should now be mounted on `/media/$USER/boot`.

Now, copy the `MLO` and `u-boot.img` files to the SD card:

```
cp MLO u-boot.img /media/$USER/boot/
sudo umount /media/$USER/boot/
```

Testing U-Boot

Insert the SD card in the board slot. To boot the board on the external micro-SD card, you need to hold the `S2` button close to the USB host port, and then power-up the board by unplugging and replugging the power cable. You can then release the `S2` button. Please beware that a warm reset performed with the reset button won't work, as it does not affect the boot source!

This seems like a very inconvenient way of booting the board, but the selection of attempting to boot from the external micro-SD card remains active across resets, until the board is ultimately powered off. So, you will just need to use the button a few times during the course.

If this is too inconvenient for you, you could use U-Boot on the external micro-SD card to flash a new version of U-Boot on the internal eMMC. This would allow you to boot without an external micro-SD card.

Here's what you should get on the serial line:

```
U-Boot SPL 2024.04 (May 22 2024 - 14:21:15 +0200)
Trying to boot from MMC1


U-Boot 2024.04 (May 22 2024 - 14:21:15 +0200)

CPU  : AM335X-GP rev 2.1
Model: TI AM335x BeagleBone Black
DRAM:  512 MiB
Core:  160 devices, 18 uclasses, devicetree: separate
WDT:   Started wdt@44e35000 with servicing every 1000ms (60s timeout)
NAND:  0 MiB
MMC:   OMAP SD/MMC: 0, OMAP SD/MMC: 1
Loading Environment from FAT... Unable to read "uboot.env" from mmc0:1...
<ethaddr> not set. Validating first E-fuse MAC
Net:   eth2: ethernet@4a100000using musb-hdrc, OUT ep1out IN ep1in STATUS ep2in
MAC de:ad:be:ef:00:01
HOST MAC de:ad:be:ef:00:00
RNDIS ready
, eth3: usb_ether
Hit any key to stop autoboot:  0
=>
```

Make sure to hit any key to prevent autoboot.

Make sure that the version and compile date are right. Otherwise, try again, because this means that you booted on the internal eMMC.

In U-Boot, type the `help` command, and explore the few commands available.

Adding a new command to the U-Boot shell

Check whether the `config` command is available. This command allows to dump the configuration settings U-Boot was compiled from.

If it's not, go back to U-Boot's configuration and enable it.

Re-run the build of U-Boot, and update the bootloader on the SD card and test that the command is now available and works as expected.

Playing with the U-Boot environment

Display the U-Boot environment using `printenv`.

Set a new U-Boot variable `foo` to a value of your choice, using `setenv`, and verify it has been set. Reset the board, and check if `foo` is still defined: it should not.

Now repeat this process, but before resetting the board, use `saveenv`. After the reset, check the `foo` variable is still defined.

Now reset the environment to its default settings using `env default -a`, and save these changes using `saveenv`.

Setting up networking

The next step is to configure U-boot and your workstation to let your board download files, such as the kernel image and Device Tree Binary (DTB), using TFTP (Trivial File Transfer Protocol) through a network connection.

As this course supports both the BeagleBone Black and BeagleBone Black Wireless boards, we're keeping things simple by using Ethernet over USB device as this works for both boards (as the Wireless board has no native Ethernet port). So, networking will work through the USB device cable that is already used to power up the board.

Caution: For the following to work, make sure that your board is powered by a USB port on your PC. Otherwise, networking over USB cannot work.

Network configuration on the target

Let's configure networking in U-Boot:

- `ipaddr`: IP address of the board
- `serverip`: IP address of the PC host

```
=> setenv ipaddr 192.168.0.100
=> setenv serverip 192.168.0.1
```

Of course, make sure that this address belongs to a separate network segment from the one of the main company network.

We also need to configure Ethernet over USB device:

- `ethprime`: controls which interface gets used first
- `usbnet_devaddr`: MAC address on the device side
- `usbnet_hostaddr`: MAC address on the host side

```
=> setenv ethprime usb_ether
=> setenv usbnet_devaddr f8:dc:7a:00:00:02
=> setenv usbnet_hostaddr f8:dc:7a:00:00:01
```

To make these settings permanent, save the environment:

 © 2004-2026 Bootlin, CC BY-SA license

```
=> saveenv
```

Network configuration on the PC host

To configure your network interface on the workstation side, we need to know the name of the network interface connected to your board.

Note that when the board is waiting at the U-Boot prompt, no network interface will show up on the workstation side. It is only when U-Boot is actively executing a network-related command (such as `ping` or `tftp`) that it brings up the USB network connection.

From the board, run `ping 192.168.0.1`, and while the `ping` command is running, you should see on your workstation a new network interface named `enx<macaddr>`. Given the value we gave to `usbnet_hostaddr`, it will therefore be `enxf8dc7a000001`. Note that pinging the board from your PC will not work: when U-Boot is waiting at its prompt, it is not able to reply to ping requests.



Then, instead of configuring the host IP address from NetworkManager's graphical interface, let's do it through its command line interface, which is so much easier to use:

```
nmcli con add type ethernet ifname enxf8dc7a000001 ip4 192.168.0.1/24
```

Setting up the TFTP server

Let's install a TFTP server on your development workstation:

```
sudo apt install tftpd-hpa
```

You can then test the TFTP connection. First, put a small text file in the directory exported through TFTP on your development workstation. Then, from U-Boot, do:

```
=> tftp 0x81000000 textfile.txt
```



In case the download fails, make sure your host interface is correctly configured and if a firewall is enabled make sure it does not filter out our requests:

```
sudo ufw allow from 192.168.0.100
```

Otherwise the `tftp` command should have downloaded the `textfile.txt` file from your development workstation into the board's memory at location `0x81000000`[5].

You can verify that the download was successful by dumping the contents of the memory:

```
=> md 0x81000000
```

We will see in the next labs how to use U-Boot to download, flash and boot a kernel.

Rescue binaries

If you have trouble generating binaries that work properly, or later make a mistake that causes you to lose your bootloader binaries, you will find working versions under `data/` in the current lab directory.

[5] This location is part of the board DRAM. If you want to check where this value comes from, you can check the SoC datasheet at https://www.ti.com/lit/ug/spruh73q/spruh73q.pdf. It's a big document (more than 5,000 pages). In this document, look for `ARM Cortex-A8 Memory Map` and you will find the SoC memory map. You will see that the address range for the memory controller (*EMIF0 SDRAM*) starts at the address we are looking for. You can also try with other values in the RAM address range.

Fetching Linux kernel sources

Objective: learn how to fetch the Linux kernel sources from git, from both the master and stable branches.

After this lab, you will be able to:

- Get the kernel sources from git, using the official Linux source tree.
- Fetch the sources for the stable Linux releases, by declaring a remote tree and getting stable branches from it.

Setup

Create the `$HOME/embedded-linux-bbb-labs/kernel` directory and go into it.

Since the Linux kernel git repository is huge, our goal here is to start downloading it right now, before starting the lectures about the Linux kernel.

Cloning the mainline Linux tree

To begin working with the Linux kernel sources, we need to clone its reference git tree, the one managed by Linus Torvalds.

However, this requires downloading more than 2.8 GB of data. If you are running this command from home, or if you have very fast access to the Internet at work (and if you are not 256 participants in the training room), you can do it directly by connecting to https://git.kernel.org:

```
git clone https://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux
cd linux
```

If Internet access is not fast enough and if multiple people have to share it, your instructor will give you a USB flash drive with a `tar.gz` archive of a recently cloned Linux source tree.

You will just have to extract this archive in the current directory, and then pull the most recent changes over the network:

```
tar xf linux-git.tar.gz
cd linux
git checkout master
git pull
```

Of course, if you directly ran `git clone`, you won't have to run `git pull`, as `git clone` already retrieved the latest changes. You may need to run `git pull` in the future though, if you want to update a newer Linux version.

Accessing stable releases

The Linux kernel repository from Linus Torvalds contains all the main releases of Linux, but not the stable versions: they are maintained by a separate team, and hosted in a separate repository.

We will add this separate repository as another *remote* to be able to use the stable releases:

```
git remote add stable https://git.kernel.org/pub/scm/linux/kernel/git/stable/linux
git fetch stable
```



 © 2004-2026 Bootlin, CC BY-SA license

As this still represents many git objects to download (2.4 GiB when 6.9 was the latest version), if you are using an already downloaded git tree, your instructor will probably have fetched the *stable* branch ahead of time for you too. You can check by running:

```
git branch -a
```

We will choose a particular stable version in the next labs.

Now, let's continue the lectures. This will leave time for the commands that you typed to complete their execution (if needed).

© 2004-2026 Bootlin, CC BY-SA license

Kernel - Cross-compiling

Objective: Learn how to cross-compile a kernel for an ARM target platform.

After this lab, you will be able to:

- Checkout a stable version of the Linux kernel
- Set up a cross-compiling environment
- Cross compile the kernel for the BeagleBone Black
- Use U-Boot to download the kernel
- Check that the kernel you compiled starts the system

Setup

Stay in the `$HOME/embedded-linux-bbb-labs/kernel` directory.

Choose a particular stable version of Linux

We will use `linux-6.6.x`, which corresponds to an LTS release, and which this lab was tested with.

First, let's get the list of branches we have available:

```
$ cd linux
$ git branch -a
```

As we will do our labs with the Linux 6.6, the remote branch we are interested in is `remotes/stable/linux-6.6.y`.

First, execute the following command to check which version you currently have:

```
$ make kernelversion
```

You can also open the `Makefile` and look at the beginning of it to check this information.

Now, let's create a local branch starting from that remote branch:

```
$ git checkout linux-6.6.y
```

Check the version again using the `make kernelversion` command to make sure you now have a 6.6.x version.

Cross-compiling environment setup

To cross-compile Linux, you need to have a cross-compiling toolchain. We will use the cross-compiling toolchain that we previously produced, so we just need to make it available in the PATH:

```
$ export PATH=$HOME/x-tools/arm-training-linux-musleabihf/bin:$PATH
```

Also, don't forget to either:

- Define the value of the `ARCH` and `CROSS_COMPILE` variables in your environment (using `export`)
- **Or** specify them on the command line at every invocation of `make`, i.e.: `make ARCH=... CROSS_COMPILE= ... <target>`



It is assumed that the `ARCH` and `CROSS_COMPILE` variables have been exported.

 © 2004-2026 Bootlin, CC BY-SA license

Install dependencies

To build the Linux kernel, you need to install the `bc` utility:

```
$ sudo apt install bc
```

Linux kernel configuration

By running `make help`, look for the proper Makefile target to configure the kernel for your processor.

In our case look for a configuration for boards in the OMAP2 and later family which the AM335x found in the BeagleBone belongs to.



So, apply this configuration by running `make <default-configuration>`, and then run `make menuconfig` to fine-tune it.

- Disable `CONFIG_GCC_PLUGINS` if it is set. This will skip building special *gcc* plugins, which would require extra dependencies for the build.
- Add options to support USB host and networking over USB device:
  - `CONFIG_USB=y`
  - `CONFIG_USB_GADGET=y`
  - `CONFIG_USB_MUSB_HDRC=y` *Driver for the USB OTG controller*
  - `CONFIG_USB_MUSB_DSPS=y`
  - `CONFIG_USB_MUSB_DUAL_ROLE=y` *Use the USB OTG both in host and device (gadget) modes. This will be needed later to use the board's USB host port.*
  - Check the dependencies of `CONFIG_AM335X_PHY_USB` and find the way to set `CONFIG_AM335X_PHY_USB=y`
  - Find the "USB Gadget precomposed configurations" menu and set it to *static* instead of *module* so that `CONFIG_USB_ETH=y`
- Also compile your kernel with `CONFIG_INPUT_EVDEV=y`, to have the same default setting as in our labs with the STM32MP1 boards.



Cross compiling

You're now ready to cross-compile your kernel. Simply run:

```
$ make
```

and wait a while for the kernel to compile. Don't forget to use `make -j<n>` if you have multiple cores on your machine!

Look at the kernel build output to see which file contains the kernel image.

Also look in the Device Tree Source directory to see which `.dtb` files got compiled. Find which `.dtb` file corresponds to your board.

For the BeagleBone Black, its `am335x-boneblack.dtb`, and for the BeagleBone Black Wireless, its `am335x-boneblack-wireless.dtb`.

Load and boot the kernel using U-Boot

As we are going to boot the Linux kernel from U-Boot, we need to set the `bootargs` environment corresponding to the Linux kernel command line:

```
=> setenv bootargs console=ttyS0,115200n8
```

```
=> saveenv
```

We will use TFTP to load the kernel image on the board:

- On your workstation, copy the `zImage` and DTB (`am335x-boneblack.dtb`) to the directory exposed by the TFTP server.
- On the target (in the U-Boot prompt), load `zImage` from TFTP into RAM:

  ```
  => tftp 0x81000000 zImage
  ```

- Now, also load the DTB file into RAM:

  ```
  => tftp 0x82000000 am335x-boneblack.dtb
  ```

- Boot the kernel with its device tree:

  ```
  => bootz 0x81000000 - 0x82000000
  ```

You should see Linux boot and finally panicking. This is expected: we haven't provided a working root filesystem for our device yet.

You can now automate all this every time the board is booted or reset. Reset the board, and customize `bootcmd`:

```
=> setenv bootcmd 'tftp 0x81000000 zImage; tftp 0x82000000 am335x-boneblack.dtb; bootz
    0x81000000 - 0x82000000'
=> saveenv
```

Known issue: with at least U-Boot 2023.04 and 2024.04 and with USB networking, there is an issue running two `tftp` commands in a row in `bootcmd`. To work around this issue before we get a chance to fix it upstream, insert `sleep 0.1` between the two `tftp` commands and everything will work as expected.

Restart the board to make sure that booting the kernel is now automated.

 © 2004-2026 Bootlin, CC BY-SA license

Tiny embedded system with BusyBox

Objective: making a tiny yet full featured embedded system

After this lab, you will:

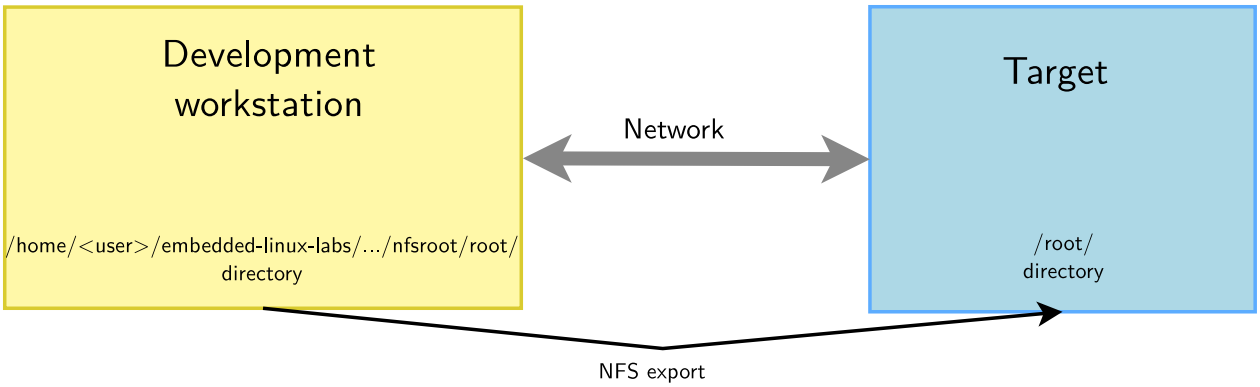
- be able to configure and build a Linux kernel that boots on a directory on your workstation, shared through the network by NFS.
- be able to create and configure a minimalistic root filesystem from scratch (ex nihilo, out of nothing, entirely hand made...) for your target board.
- understand how small and simple an embedded Linux system can be.
- be able to install BusyBox on this filesystem.
- be able to create a simple startup script based on /sbin/init.
- be able to set up a simple web interface for the target.

Lab implementation

While (s)he develops a root filesystem for a device, a developer needs to make frequent changes to the filesystem contents, like modifying scripts or adding newly compiled programs.

It isn't practical at all to reflash the root filesystem on the target every time a change is made. Fortunately, it is possible to set up networking between the development workstation and the target. Then, workstation files can be accessed by the target through the network, using NFS.

Unless you test a boot sequence, you no longer need to reboot the target to test the impact of script or application updates.



Setup

Go to the `$HOME/embedded-linux-bbb-labs/tinysystem/` directory.

Kernel configuration

We will reuse the kernel sources from our previous lab, in `$HOME/embedded-linux-bbb-labs/kernel/`.

In the kernel configuration built in the previous lab, verify that you have all options needed for booting the system using a root filesystem mounted over NFS. Also check that `CONFIG_DEVTMPFS_MOUNT` is enabled (we will explain it later in this lab). If necessary, rebuild your kernel.

Setting up the NFS server

Create a `nfsroot` directory in the current lab directory. This `nfsroot` directory will be used to store the contents of our new root filesystem.

Install the NFS server by installing the `nfs-kernel-server` package if you don't have it yet. Once installed, edit the `/etc/exports` file as root to add the following line, assuming that the IP address of your board will be `192.168.0.100`:



```
/home/<user>/embedded-linux-bbb-labs/tinysystem/nfsroot 192.168.0.100(rw,no_root_squash,
no_subtree_check)
```

Of course, replace `<user>` by your actual user name.

Make sure that the path and the options are on the same line. Also make sure that there is no space between the IP address and the NFS options, otherwise default options will be used for this IP address, causing your root filesystem to be read-only.

Then, make the NFS server use the new configuration:

```
$ sudo exportfs -r
```

Booting the system

First, boot the board to the U-Boot prompt. Before booting the kernel, we need to tell it that the root filesystem should be mounted over NFS, by setting some kernel parameters.

So add settings to the `bootargs` environment variable, **in just 1 line**:



```
=> setenv bootargs ${bootargs} root=/dev/nfs ip=192.168.0.100:::::usb0
   g_ether.dev_addr=f8:dc:7a:00:00:02 g_ether.host_addr=f8:dc:7a:00:00:01
   nfsroot=192.168.0.1:/home/<user>/embedded-linux-bbb-labs/tinysystem/nfsroot,nfsvers=3,tcp rw
```

Once again, replace `<user>` by your actual user name.

Of course, you need to adapt the IP addresses to your exact network setup. Save the environment variables (with `saveenv`).

Now, boot your system. The kernel should be able to mount the root filesystem over NFS:

```
VFS: Mounted root (nfs filesystem) on device X:Y.
```

If the kernel fails to mount the NFS filesystem, look carefully at the error messages in the console. If this doesn't give any clue, you can also have a look at the NFS server logs in `/var/log/syslog`.

However, at this stage, the kernel should stop because of the below issue:

```
[    7.476715] devtmpfs: error mounting -2
```

This happens because the kernel is trying to mount the *devtmpfs* filesystem in `/dev/` in the root filesystem. This virtual filesystem contains device files (such as `ttyS0`) for all the devices known to the kernel, and with `CONFIG_DEVTMPFS_MOUNT`, our kernel tries to automatically mount *devtmpfs* on `/dev`.

To address this, just create a `dev` directory under `nfsroot` and reboot.

Now, the kernel should complain for the last time, saying that it can't find an init application:

```
Kernel panic - not syncing: No working init found.  Try passing init= option to
kernel. See Linux Documentation/admin-guide/init.rst for guidance.
```

Obviously, our root filesystem being mostly empty, there isn't such an application yet. In the next paragraph, you will add BusyBox to your root filesystem and finally make it usable.

 © 2004-2026 Bootlin, CC BY-SA license

Root filesystem with BusyBox

Download the sources of the latest BusyBox 1.37.x release:



```
git clone git://busybox.net/busybox.git
cd busybox/
git checkout 1_37_stable
```

Now, configure BusyBox with the configuration file provided in the `data/` directory (remember that the BusyBox configuration file is `.config` in the BusyBox sources).



Then, you can use `$ make menuconfig` to further customize the BusyBox configuration. At least, keep the setting that builds a static BusyBox. Compiling BusyBox statically in the first place makes it easy to set up the system, because there are no dependencies on libraries. Later on, we will set up shared libraries and recompile BusyBox.

If you are running on a distribution that uses GCC >= 14.x, you will face an issue when trying to run `make menuconfig`, caused by a bug in Busybox, unfixed as of Busybox 1.37.0. You can fix this issue by applying an additional patch to the Busybox source:

```
git am $HOME/embedded-linux-bbb-labs/tinysystem/data/\
    0001-menuconfig-GCC-failing-saying-ncurses-is-not-found.patch
```

Prior to building Busybox, make sure your `CROSS_COMPILE` environment is set to the correct value, pointing to the toolchain we have previously compiled, and used to build our bootloader and Linux kernel.

Build BusyBox:

```
$ make
```

Going back to the BusyBox configuration interface, check the installation directory for BusyBox[6]. Set it to the path to your `nfsroot` directory.

Now run `$ make install` to install BusyBox in this directory.

Try to boot your new system on the board. You should now reach a command line prompt, allowing you to execute the commands of your choice.



Virtual filesystems

Run the `# ps` command. You can see that it complains that the `/proc` directory does not exist. The ps command and other process-related commands use the `proc` virtual filesystem to get their information from the kernel.

From the Linux command line in the target, create the `proc`, `sys` and `etc` directories in your root filesystem.

Now mount the `proc` virtual filesystem. Now that `/proc` is available, test again the `ps` command.

Note that you can also now halt your target in a clean way with the `halt` command, thanks to `proc` being mounted[7].

System configuration and startup

The first user space program that gets executed by the kernel is `/sbin/init` and its configuration file is `/etc/inittab`.

In the BusyBox sources, read details about `/etc/inittab` in the `examples/inittab` file.

[6] You will find this setting in `Settings -> Install Options -> Destination path for 'make install'`.

[7] `halt` can find the list of mounted filesystems in `/proc/mounts`, and unmount each of them in a clean way before shutting down.

Then, create a `/etc/inittab` file and a `/etc/init.d/rcS` startup script declared in `/etc/inittab`. In this startup script, mount the `/proc` and `/sys` filesystems.

Any issue after doing this?



Starting the shell in a proper terminal

Before the shell prompt, you probably noticed the below warning message:

```
/bin/sh: can't access tty; job control turned off
```

This happens because the shell specified in the `/etc/inittab` file in started by default in `/dev/console`:

```
::askfirst:/bin/sh
```

When nothing is specified before the leading `::`, `/dev/console` is used. However, while this device is fine for a simple shell, it is not elaborate enough to support things such as job control (`[Ctrl][c]` and `[Ctrl][z]`), allowing to interrupt and suspend jobs.

So, to get rid of the warning message, we need `init` to run `/bin/sh` in a real terminal device:

```
ttyS0::askfirst:/bin/sh
```

Reboot the system and the message will be gone!

Switching to shared libraries

Take the `hello.c` program supplied in the lab `data` directory. Cross-compile it for ARM, dynamically-linked with the libraries[8], and run it on the target.



You will first encounter a very misleading `not found` error, which is not because the `hello` executable is not found, but because something else was not found while trying to execute this executable.

You can find it by running `file hello` on the host:

```
hello: ELF 32-bit LSB executable, ARM, EABI5 version 1 (SYSV),
dynamically linked, interpreter /lib/ld-musl-armhf.so.1, not stripped
```

So, what's missing is the `/lib/ld-musl-armhf.so.1` executable, which is the dynamic linker required to execute any program compiled with shared libraries. Using the `find` command, look for this file in the toolchain install directory, and copy it to the `lib/` directory on the target.

Then, running the executable again and see that the loader executes and finds out which shared libraries are missing.

In our case with the Musl C library, the dynamic linker also contains the C library, so the program should execute fine, as no further shared libraries are required.

If you still get the same error message, just try again a few seconds later. Such a delay can be needed because the NFS client can take a little time (at most 30-60 seconds) before seeing the changes made on the NFS server.

Now that the small test program works, we are going to recompile BusyBox without the static compilation option, so that BusyBox takes advantage of the shared libraries that are now present on the target.

Before doing that, measure the size of the `busybox` executable.



Then, build BusyBox with shared libraries, and install it again on the target filesystem. Make sure that the system still boots and see how much smaller the `busybox` executable got.

[8] Invoke your cross-compiler in the same way you did during the toolchain lab

 © 2004-2026 Bootlin, CC BY-SA license

Implement a web interface for your device

Replicate `data/www/` to the `/www` directory in your target root filesystem.



Now, run the BusyBox http server from the target command line:

```
=> /usr/sbin/httpd -h /www/
```

It will automatically background itself.

If you use a proxy, configure your host browser so that it doesn't go through the proxy to connect to the target IP address, or simply disable proxy usage. Now, test that your web interface works well by opening `http://192.168.0.100/index.html` on the host.

See how the dynamic pages are implemented. Very simple, isn't it?

Finish by adding the command that starts the web server to your startup script, so that it is always started on your target.



Going further

If you have time before the others complete their labs...

Initramfs booting

Configure your kernel to include the contents of the `nfsroot` directory as an initramfs.

Before doing this, you will need to create an `init` link in the toplevel directory to `sbin/init`, because the kernel will try to execute `/init`.

You will also need to mount *devtmpfs* from the `rcS` script, it cannot be mounted automatically by the kernel when you're booting from an initramfs.

Note: you won't need to modify your `root=` setting in the kernel command line. It will just be ignored if you have an initramfs.

When this works, go back to booting the system through NFS. This will be much more convenient in the next labs.

© 2004-2026 Bootlin, CC BY-SA license

Accessing Hardware Devices

Objective: learn how to access hardware devices and declare new ones.

Goals

Now that we have access to a command line shell thanks to a working root filesystem, we can now explore existing devices and make new ones available. In particular, we will make changes to the Device Tree and compile an out-of-tree Linux kernel module.

Setup

Go to the `$HOME/embedded-linux-bbb-labs/hardware` directory, which provides useful files for this lab.

However, we will go on booting the system through NFS, using the root filesystem built by the previous lab.

Exploring /dev

Start by exploring `/dev` on your target system. Here are a few noteworthy device files that you will see:

- *Terminal devices*: devices starting with `tty`. Terminals are user interfaces taking text as input and producing text as output, and are typically used by interactive shells. In particular, you will find `console` which matches the device specified through `console=` in the kernel command line. You will also find the `ttyS0` device file.
- *Pseudo-terminal devices*: devices starting with `pty`, used when you connect through SSH for example. Those are virtual devices, but there are so many in `/dev` that we wanted to give a description here.
- MMC device(s) and partitions: devices starting with `mmcblk`. You should here recognize the MMC device(s) on your system and the associated partitions.
- If you have a real board (not QEMU) and a USB stick, you could plug it in and if your kernel was built with USB host and mass storage support, you should see a new `sda` device appear, together with the `sda<n>` devices for its partitions.

Don't hesitate to explore `/dev` on your workstation too and ask any questions to your instructor.

Exploring /sys

The next thing you can explore is the *Sysfs* filesystem.

A good place to start is `/sys/class`, which exposes devices classified by the kernel frameworks which manage them.

For example, go to `/sys/class/net`, and you will see all the networking interfaces on your system, whether they are internal, external or virtual ones.

Find which subdirectory corresponds to the network connection to your host system, and then check device properties such as:

- `speed`: will show you whether this is a gigabit or hundred megabit interface.
- `address`: will show the device MAC address. No need to get it from a complex command!
- `statistics/rx_bytes` will show you how many bytes were received on this interface.

Don't hesitate to look for further interesting properties by yourself!

 © 2004-2026 Bootlin, CC BY-SA license

You can also check whether `/sys/class/thermal` exists and is not empty on your system. That's the thermal framework, and it allows to access temperature measures from the thermal sensors on your system.

Next, you can now explore all the buses (virtual or physical) available on your system, by checking the contents of `/sys/bus`.

In particular, go to `/sys/bus/mmc/devices` to see all the MMC devices on your system. Go inside the directory for the first device and check several files (for example):

- `serial`: the serial number for your device.
- `preferred_erase_size`: the preferred erase block for your device. It's recommended that partitions start at multiples of this size.
- `name`: the product name for your device. You could display it in a user interface or log file, for example.
- `date`: apparently the manufacturing date for the device.

Don't hesitate to spend more time exploring `/sys` on your system and asking questions to your instructor.

Driving GPIOs

At this stage, we can only explore GPIOs through the legacy interface in `/sys/class/gpio`, because the *libgpiod* interface commands are provided through a dedicated project which we have to build separately, and *Busybox* does not provide a re-implementation for the *libgpiod* tools. In a later lab, we will build *libgpiod* tools which use the modern `/dev/gpiochipX` interface.

The first thing to do is to enable this legacy interface by enabling `CONFIG_GPIO_SYSFS` in the kernel configuration. Also make sure *Debugfs* is enabled (`CONFIG_DEBUG_FS` and `CONFIG_DEBUG_FS_ALLOW_ALL`).

After rebooting the new kernel, the first thing to do is to mount the *Debugfs* filesystem:

```
# mount -t debugfs debugfs /sys/kernel/debug/
```

Then, you can check information about available GPIOs banks and which GPIOs are already in use:

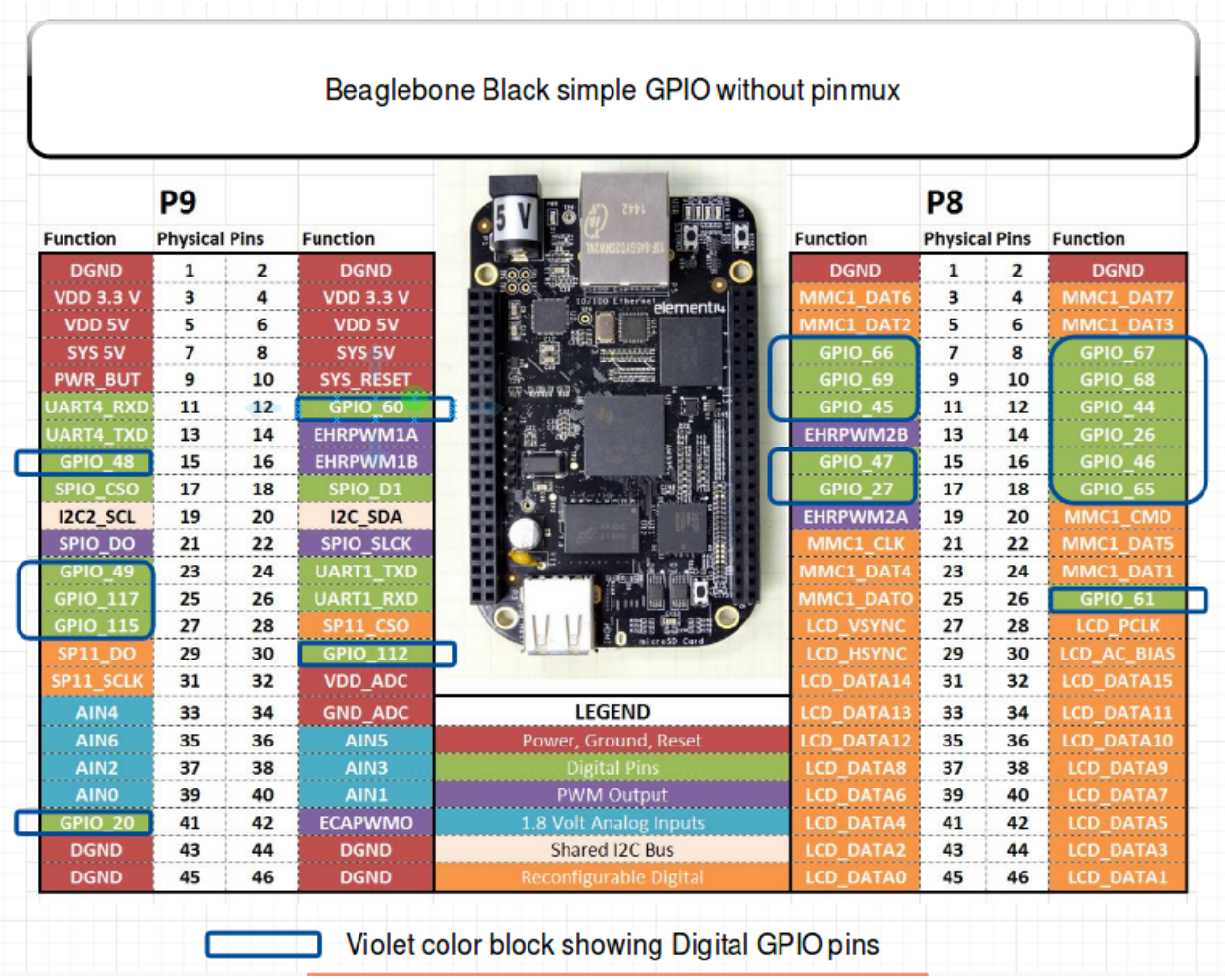
```
# cat /sys/kernel/debug/gpio
```

We are going to use one of the free GPIOs on the expansion headers of the board, which is not already used by another device.

Take one of the M-M breadboard wires provided by your instructor and:

- Connect one end to pin 12 of connector P9
- Connect the other end to pin 1 (`DGND`) of connector P9

© 2004-2026 Bootlin, CC BY-SA license



Source: https://elinux.org/File:BBB_I-O_pins_.png

If you check the description of the P9 connector on the board System Reference Manual, you can see that pin 12 is now called `GPIO1_28` instead of `GPIO_60` in the above diagram. This pin is already configured as a GPIO by default (no need to change pin muxing to use this pin as a GPIO).

If you get back to the contents of `/sys/kernel/debug/gpio`, you'll recognize the association between `gpio-28` on GPIO pin bank 0 (`gpiochip0`) and header pin `P9_12`. That's very useful information, but you don't have this level of details for all boards, unfortunately.

We now have everything we need to drive this GPIO using the legacy interface. First, let's enable it:

```
# cd /sys/class/gpio
# echo 540 > export
```

If indeed the pin is still available, this should create a new `gpio540` file in `/sys/class/gpio`.

We can now configure this pin as input:

```
# echo in > gpio540/direction
```

And check its value:

```
# cat gpio540/value
0
```

 © 2004-2026 Bootlin, CC BY-SA license

The value should be `0` as the pin is connected to a ground level.

Now, let's connect our GPIO pin to pin 3 (`VDD 3.3V`) of connector P9. Check the above diagram if needed.

Let's check the value again:

```
# cat gpio540/value
1
```

The value is `1` because our pin is connected to a 3.3V level now.

You could use this GPIO to add a button switch to your board, for example.

Note that you could also configure the pin as output and set its value through the `value` file. This way, you could add an external LED to your board, for example.

Before moving on to the next section, you can also check `/sys/kernel/debug/gpio` again, and see that `gpio-540` is now in use, through the sysfs interface, and is configured as an input pin.

When you're done, you can see your GPIO free:

```
# echo 540 > unexport
```

Driving LEDs

First, make sure your kernel is compiled with `CONFIG_LEDS_CLASS=y`, `CONFIG_LEDS_GPIO=y` and `CONFIG_LEDS_TRIGGER_TIMER=y`.

Then, go to `/sys/class/leds` to see all the LEDs that you are allowed to control.

Let's control the LED which is called `beaglebone:green:heartbeat`.

Go into the directory for this LED, and check its trigger (what routine is used to drive its value):

```
# cat trigger
```

As you can see, there are many triggers to choose from. You can disable all triggers by:

```
# echo none > trigger
```

And then directly control the LED:

```
# echo 1 > brightness
# echo 0 > brightness
```

You could also use the `timer` trigger to light the LED with specified time on and time off:

```
# echo timer > trigger
# echo 10 > delay_on
# echo 200 > delay_off
```

Managing the I2C buses and devices

Enabling an I2C bus

The next thing we want to do is connect an Nunchuk joystick to an I2C bus on our board. The I2C bus is very frequently used to connect all sorts of external devices. That's why we're covering it here.

As shown on the below picture found on `https://elinux.org/Beagleboard:Cape_Expansion_Headers`, the BeagleBone Black has two I2C busses available on its expansion headers: I2C1 and I2C2. Another one exists (I2C0), but it's not available on the external headers.

2 I2C ports

P9

DGND	1	2	DGND
VDD_3V3	3	4	VDD_3V3
VDD_5V	5	6	VDD_5V
SYS_5V	7	8	SYS_5V
PWR_BUT	9	10	SYS_RESETn
GPIO_30	11	12	GPIO_60
GPIO_31	13	14	GPIO_40
GPIO_48	15	16	GPIO_51
I2C1_SCL	17	18	I2C1_SDA
I2C2_SCL	19	20	I2C2_SDA
I2C2_SCL	21	22	I2C2_SDA
GPIO_49	23	24	I2C1_SCL
GPIO_117	25	26	I2C1_SDA
GPIO_125	27	28	GPIO_123
GPIO_121	29	30	GPIO_122
GPIO_120	31	32	VDD_ADC
AIN4	33	34	GNDA_ADC
AIN6	35	36	AIN5
AIN2	37	38	AIN3
AIN0	39	40	AIN1
GPIO_20	41	42	GPIO_7
DGND	43	44	DGND
DGND	45	46	DGND

P8

DGND	1	2	DGND
GPIO_38	3	4	GPIO_39
GPIO_34	5	6	GPIO_35
GPIO_66	7	8	GPIO_67
GPIO_69	9	10	GPIO_68
GPIO_45	11	12	GPIO_44
GPIO_23	13	14	GPIO_26
GPIO_47	15	16	GPIO_46
GPIO_27	17	18	GPIO_65
GPIO_22	19	20	GPIO_63
GPIO_62	21	22	GPIO_37
GPIO_36	23	24	GPIO_33
GPIO_32	25	26	GPIO_61
GPIO_86	27	28	GPIO_88
GPIO_87	29	30	GPIO_89
GPIO_10	31	32	GPIO_11
GPIO_9	33	34	GPIO_81
GPIO_8	35	36	GPIO_80
GPIO_78	37	38	GPIO_79
GPIO_76	39	40	GPIO_77
GPIO_74	41	42	GPIO_75
GPIO_72	43	44	GPIO_73
GPIO_70	45	46	GPIO_71

In this lab, we will try to use I2C1 on P9 pins 17 and 18, because it's more interesting to use than I2C2 which is already enabled by default.

So, let's see which I2C buses are already enabled:

```
# i2cdetect -l
i2c-2 i2c OMAP I2C adapter I2C adapter
i2c-0 i2c OMAP I2C adapter I2C adapter
```

Here you can see that I2C1 is missing.

As the bus numbering scheme in Linux doesn't always match the one on the datasheets, let's check the base addresses of the registers of these controllers:

```
# ls -l /sys/bus/i2c/devices/i2c-*
lrwxrwxrwx 1 0 Jan 1 00:59 /sys/bus/i2c/devices/i2c-0 -> ../../../devices/platform/ocp/\
    44c00000.interconnect/44c00000.interconnect:segment@200000/44e0b000.target-module/\
    44e0b000.i2c/i2c-0
lrwxrwxrwx 1 0 Jan 1 00:59 /sys/bus/i2c/devices/i2c-2 -> ../../../devices/platform/ocp/\
    48000000.interconnect/48000000.interconnect:segment@100000/4819c000.target-module/\
    4819c000.i2c/i2c-2
```

That's not completely straightforward, but you can suppose that:

- I2C0 is at address `0x44e0b000`
- I2C2 is at address `0x4819c000`

Now let's double check the addressing by looking at the TI AM335x SoC datasheet, in the `L4_WKUP Peripheral Memory Map` and `L4_PER Peripheral Memory Map` sections:

- I2C0 is indeed at address `0x44e0b000`
- I2C1 is at address `0x4802a000`
- I2C2 is indeed at address `0x4819c000`

So, we are lucky that `i2c-0` in Linux corresponds to I2C0 in the datasheet, and that `i2c-2` corresponds to I2C2. We're just missing `i2c-1`.

 © 2004-2026 Bootlin, CC BY-SA license

Customizing the Device Tree

Fortunately, I2C1 is already defined in the one of the DTS includes used by the Device Tree for our board. In our case, that's in arch/arm/boot/dts/ti/omap/am33xx-l4.dtsi. Look by yourself in this file, and you will find its definition, but with `status = "disabled";`. This means that this I2C controller is not enabled yet, and it's up to boards using it to do so.

We could modify the arch/arm/boot/dts/ti/omap/am335x-boneblack.dts file for our board, but that's not a very good idea as this file is maintained by the kernel developers. The changes that you make could collide with future changes made by the maintainers for this file.

A more futureproof idea is to create a new Device Tree file which includes the standard one, and adds custom definitions. So, create a new `arch/arm/boot/dts/ti/omap/am335x-boneblack-custom.dts` file containing:

```
/dts-v1/;
#include "am335x-boneblack.dts"

&i2c1 {
        status = "okay";
};
```

As you can see, it's also possible to include `dts` files, and not only `dtsi` ones.

Modify the arch/arm/boot/dts/ti/omap/Makefile file to add your custom Device Tree, and then have it compiled (`make dtbs`).

Reboot your board with the update.

Back to the running system, we can now see that there is one more I2C bus:

```
# i2cdetect -l
i2c-1 i2c OMAP I2C adapter I2C adapter
i2c-2 i2c OMAP I2C adapter I2C adapter
i2c-0 i2c OMAP I2C adapter I2C adapter
```

Run the below command to confirm that the new bus has the same address as in the datasheet (`0x4802a000`):

```
ls -l /sys/bus/i2c/devices/i2c-1
```

Now, let's use `i2cdetect`'s capability to probe a bus for devices. Let's start by the bus associated to `i2c-0`:

```
# i2cdetect -r 0
i2cdetect: WARNING! This program can confuse your I2C bus
Continue? [y/N] y
     0  1  2  3  4  5  6  7  8  9  a  b  c  d  e  f
00:          -- -- -- -- -- -- -- -- -- -- -- -- --
10: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
20: -- -- -- -- UU -- -- -- -- -- -- -- -- -- -- --
30: -- -- -- -- 34 -- -- -- -- -- -- -- -- -- -- --
40: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
50: 50 -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
60: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
70: -- -- -- -- -- -- -- --
```

We can see three devices on this internal bus:

- One at address `0x24`, indicated by `UU`, which means that there is a kernel driver actively driving this device.
- Two other devices at addresses `0x34` and `0x50`. We just know that they are currently not bound to a kernel driver.

Now try to probe I2C1 through `i2cdetect -r 1`.

You will see that the command will fail to connect to the bus. That's because the corresponding signals are not exposed yet to the outside connectors through pin muxing.

So, get back to your custom Device Tree and add pin muxing definitions for I2C1 (we took them from a device tree from another board with the same CPU: `arch/arm/boot/dts/ti/omap/am335x-evm.dts`) and refer to these definitions in the `i2c1` node through the `pinctrl-names` and `pinctrl-0` properties:

```
/dts-v1/;
#include "am335x-boneblack.dts"

&am33xx_pinmux {
  i2c1_pins: pinmux_i2c1_pins {
    pinctrl-single,pins = <
      AM33XX_PADCONF(AM335X_PIN_SPI0_CS0, PIN_INPUT_PULLUP, MUX_MODE2) /* spi0_cs0.i2c1_scl */
      AM33XX_PADCONF(AM335X_PIN_SPI0_D1, PIN_INPUT_PULLUP, MUX_MODE2) /* spi0_d1.i2c1_sda */
    >;
  };
};

&i2c1 {
        pinctrl-names = "default";
        pinctrl-0 = <&i2c1_pins>;
        status = "okay";
};
```

You can understand the above values thanks to the pin muxing diagram for connector P9, available at https://elinux.org/images/b/b4/HeaderP9.jpg, which was extracted from the board System Reference Manual:

PIN	PROC	NAME	MODE0	MODE1	MODE2	MODE3	MODE4	MODE5	MODE6	MODE7
1,2						GND				
3,4						DC_3.3V				
5,6						VDD_5V				
7,8						SYS_5V				
9						PWR_BUT				
10	A10	SYS_RESETn	RESET_OUT							
11	T17	UART4_RXD	gpmc_wait0	mii2_crs	gpmc_csn4	rmii2_crs_dv	mmc1_sdcd		uart4_rxd_mux2	gpio0[30]
12	U18	GPIO1_28	gpmc_be1n	mii2_col	gpmc_csn6	mmc2_dat3	gpmc_dir		mcasp0_aclkr_mux3	gpio1[28]
13	U17	UART4_TXD	gpmc_wpn	mii2_rxerr	gpmc_csn5	rmii2_rxerr	mmc2_sdcd		uart4_txd_mux2	gpio0[31]
14	U14	EHRPWM1A	gpmc_a2	mii2_txd3	rgmii2_td3	mmc2_dat1	gpmc_a18		ehrpwm1A_mux1	gpio1[18]
15	R13	GPIO1_16	gpmc_a0	gmii2_txen	rmii2_tctl	mii2_txen	gpmc_a16		ehrpwm1_tripzone_input	gpio1[16]
16	T14	EHRPWM1B	gpmc_a3	mii2_txd2	rgmii2_td2	mmc2_dat2	gpmc_a19		ehrpwm1B_mux1	gpio1[19]
17	A16	I2C1_SCL	spi0_cs0	mmc2_sdwp	I2C1_SCL	ehrpwm0_synci				gpio0[5]
18	B16	I2C1_SDA	spi0_d1	mmc1_sdwp	I2C1_SDA	ehrpwm0_tripzone				gpio0[4]
19	D17	I2C2_SCL	uart1_rtsn	timer5	dcan0_rx	I2C2_SCL	spi1_cs1			gpio0[13]
20	D18	I2C2_SDA	uart1_ctsn	timer6	dcan0_tx	I2C2_SDA	spi1_cs0			gpio0[12]
21	B17	UART2_TXD	spi0_d0	uart2_txd	I2C2_SCL	ehrpwm0B			EMU3_mux1	gpio0[3]
22	A17	UART2_RXD	spi0_sclk	uart2_rxd	I2C2_SDA	ehrpwm0A			EMU2_mux1	gpio0[2]
23	V14	GPIO1_17	gpmc_a1	gmii2_rxdv	rgmii2_rxdv	mmc2_dat0	gpmc_a17		ehrpwm0_synco	gpio1[17]
24	D15	UART1_TXD	uart1_txd	mmc2_sdwp	dcan1_rx	I2C1_SCL				gpio0[15]
25	A14	GPIO3_21*	mcasp0_ahclkx	eQEP0_strobe	mcasp0_axr3	mcasp1_axr1	EMU4_mux2			gpio3[21]
26	D16	UART1_RXD	uart1_rxd	mmc1_sdwp	dcan1_tx	I2C1_SDA				gpio0[14]
27	C13	GPIO3_19	mcasp0_fsr	eQEP0B_in	mcasp0_axr3	mcasp1_fsx	EMU2_mux2			gpio3[19]
28	C12	SPI1_CS0	mcasp0_ahclkr	ehrpwm0_synci	mcasp0_axr2	spi1_cs0	eCAP2_in_PWM2_out			gpio3[17]
29	B13	SPI1_D0	mcasp0_fsx	ehrpwm0B		spi1_d0	mmc1_sdcd_mux1			gpio3[15]
30	D12	SPI1_D1	mcasp0_axr0	ehrpwm0_tripzone		spi1_d1	mmc2_sdcd_mux1			gpio3[16]
31	A13	SPI1_SCLK	mcasp0_aclkx	ehrpwm0A		spi1_sclk	mmc0_sdcd_mux1			gpio3[14]
32						VADC				
33	C8					AIN4				
34						AGND				
35	A8					AIN6				
36	B8					AIN5				
37	B7					AIN2				
38	A7					AIN3				
39	B6					AIN0				
40	C7					AIN1				
41#	D14	CLKOUT2	xdma_event_intr1		tclkin	clkout2	timer7_mux1		EMU3_mux0	gpio0[20]
	D13	GPIO3_20	mcasp0_axr1	eQEP0_index		Mcasp1_axr0	emu3			gpio3[20]
42@	C18	GPIO0_7	eCAP0_in_PWM0_out	uart3_txd	spi1_cs1	pr1_ecap0_ecap_capin_apwm_o	spi1_sclk	mmc0_sdwp	xdma_event_intr2	gpio0[7]
	B12	GPIO3_18	Mcasp0_aclkr	eQEP0A_in	Mcaspo_axr2	Mcasp1_aclkx				gpio3[18]
43-46						GND				

- `AM335X_PIN_SPI0_CS0` and `AM335X_PIN_SPI0_D1` are the offsets of the registers controlling pin muxing for the corresponding pins of the SoC package.

 © 2004-2026 Bootlin, CC BY-SA license

- MUX_MODE2 corresponds to MODE2, to get I2C1 SCL and SDA signals on such pins.
- PIN_INPUT_PULLUP is one of the supported options for these pins.

Recompile your Device Tree and reboot.

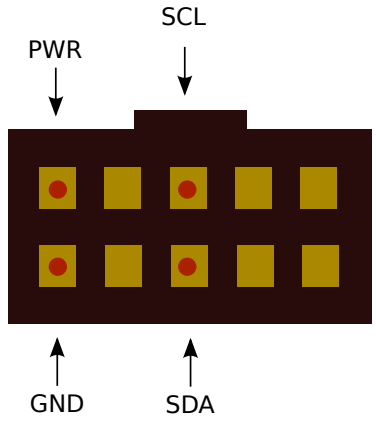
You should now be able to probe your bus:

```
# i2cdetect -r 1
i2cdetect: WARNING! This program can confuse your I2C bus
Continue? [y/N] y
     0  1  2  3  4  5  6  7  8  9  a  b  c  d  e  f
00:          -- -- -- -- -- -- -- -- -- -- -- -- --
10: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
20: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
30: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
40: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
50: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
60: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
70: -- -- -- -- -- -- -- --
```

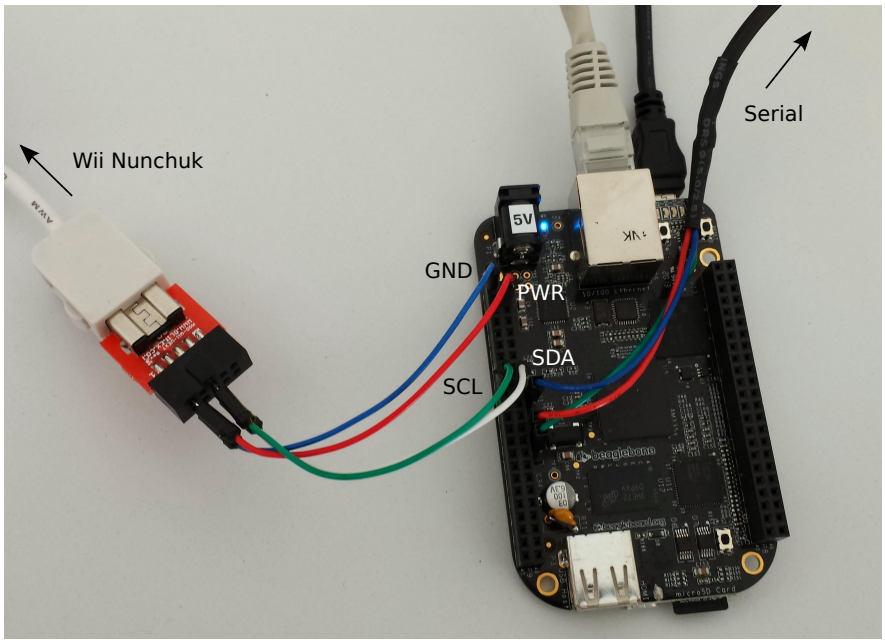
No device is detected yet, because this bus is just used for external devices. It's time to add one though.

Adding and enabling an I2C device

Let's connect the Nunchuk provided by your instructor to the I2C1 bus on the board, using breadboard wires:



Nunchuk i2c pinout
(UEXT connector from Olimex, front view)



- Connect the Nunchuk PWR pin to pin 4 (3V3) of connector P9
- Connect the Nunchuk GND pin to pin 1 (GND) of connector P9
- Connect the Nunchuk SCL pin to pin 17 of connector P9
- Connect the Nunchuk SDA pin to pin 18 of connector P9

If you didn't do any mistake, your new device should be detected at address 0x52:

```
# i2cdetect -r 1
```

```
i2cdetect: WARNING! This program can confuse your I2C bus
Continue? [y/N] y
     0  1  2  3  4  5  6  7  8  9  a  b  c  d  e  f
00:          -- -- -- -- -- -- -- -- -- -- -- -- --
10: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
20: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
30: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
40: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
50: -- -- 52 -- -- -- -- -- -- -- -- -- -- -- -- --
60: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
70: -- -- -- -- -- -- -- --
```

We will later compile an out-of-tree kernel module to support this device.

Plugging a USB audio headset

In the next labs, we are going to play audio using a USB audio headset. Let's see whether our kernel supports such hardware by plugging the headset provided by your instructor.

Before plugging the device, look at the output of `lsusb`:

```
# lsusb
Bus 001 Device 001: ID 1d6b:0002
```

Now, when you plug the USB headset, a number of messages should appear on the console, and running `lsusb` again should show an additional device:

```
# lsusb
Bus 001 Device 001: ID 1d6b:0002
Bus 001 Device 003: ID 0d8c:0014
```

The device of vendor ID `0d8c` and product ID `0014` has appeared. Of course, this depends on the actual USB audio device that you used.

The device also appears in `/sys/bus/usb/devices/`, in a directory whose name depends on the topology of the USB bus. When the device is plugged in the kernel messages show:

```
usb 1-1: new full-speed USB device number 4 using musb-hdrc
```

So if we go in `/sys/bus/usb/devices/1-1`, we get the *sysfs* representation of this USB device:

```
# cd /sys/bus/usb/devices/1-1
# cat idVendor
0d8c
# cat idProduct
0014
# cat manufacturer
C-Media Electronics Inc.
# cat product
USB Audio Device
```

However, while the USB device is detected, we currently do not have any driver for this device, so no actual sound card is detected.

Enabling, installing and using in-tree kernel modules

Go back to the kernel source directory.

The Linux kernel has a generic driver supporting all USB audio devices supporting the standard USB audio

 © 2004-2026 Bootlin, CC BY-SA license

class. This driver can be enabled using the CONFIG_SND_USB_AUDIO configuration option. Look for this parameter in the kernel configuration, and you should find that it is already enabled as a module.

So, instead of compiling the corresponding driver as a built-in, that's a good opportunity to practice with kernel modules.

So, compile your modules:

```
make modules
```

Then, following details given in the lectures, install the modules in our NFS root filesystem (`$HOME/embedded-linux-bbb-labs/tinysystem/nfsroot`).

Also make sure to update the kernel image (`make zImage`), and reboot the board. Indeed, due to the changes we have made to the kernel source code, the kernel version is now `6.6.<x>-dirty`, the *dirty* keyword indicating that the Git working tree has uncommitted changes. The modules are therefore installed in `/lib/modules/6.6.<x>-dirty/`, and the version of the running Linux kernel must match this.

After rebooting, try to load the module that we need:

```
modprobe snd-usb-audio
```

By running `lsmod`, see all the module dependencies that were loaded too.

You can also see that a new USB device driver in `/sys/bus/usb/drivers/snd-usb-audio`. This directory shows which USB devices are bound to this driver.

You can check that `/proc/asound` now exists (thanks to loading modules for ALSA, the Linux sound subsystem), and that one sound card is available:

```
# cat /proc/asound/cards
 0 [Device ]: USB-Audio - USB Audio Device
                      GeneralPlus USB Audio Device at usb-musb-hdrc.1-1, full speed
```

Check also the `/dev/snd` directory, which should now contain some character device files. These will be used by the user-space libraries and applications to access the audio devices.

Modify your startup scripts so that the `snd-usb-audio` module is always loaded at startup.

We cannot test the sound card yet, as we will need to build some software first. Be patient, this is coming soon.

Compiling and installing an out-of-tree kernel module

The next device we want to support is the I2C Nunchuk. There is a driver in the kernel to support it when connected to a Wiimote controller, but there is no such driver to support it as an I2C device.

Fortunately, one is provided in `$HOME/embedded-linux-bbb-labs/hardware/data/nunchuk/nunchuk.c`. You can check Bootlin's Linux kernel and driver development course to learn how to implement all sorts of device drivers for Linux.

Go to this directory, and compile the out-of-tree module as follows:

```
make -C $HOME/embedded-linux-bbb-labs/kernel/linux M=$PWD
```

Here are a few explanations:

- The `-C` option lets `make` know which Makefile to use, here the toplevel Makefile in the kernel sources.
- `M=$PWD` tells the kernel Makefile to build external module(s) from the file(s) in the current directory.

Now, you can install the compiled module in the NFS root filesystem by passing the `modules_install` target and specifying the target directory through the `INSTALL_MOD_PATH` variable:

```
make -C $HOME/embedded-linux-bbb-labs/kernel/linux \
     M=$PWD \
     INSTALL_MOD_PATH=$HOME/embedded-linux-bbb-labs/tinysystem/nfsroot \
     modules_install
```

You can see that this installs out-of-tree kernel modules under `lib/modules/<version>/updates/`.

Back on the target, you can now check that your custom module can be loaded:

```
# modprobe nunchuk
[ 4317.737978] nunchuk: loading out-of-tree module taints kernel.
```

See kbuild/modules in kernel documentation for details about building out-of-tree kernel modules.

However, run `i2cdetect -r 1` again. You will see that the Nunchuk is still detected, but still not driven by the kernel. Otherwise, it would be signaled by the `UU` character. You may also look at the `nunchuk.c` file and notice a `Nunchuk device probed successfully` message that you didn't see when loading the module.

That's because the Linux kernel doesn't know about the Nunchuk device yet, even though the driver for this kind of devices is already loaded. Our device also has to be described in the Device Tree.

You can confirm this by having a look at the contents of the `/sys/bus/i2c` directory. It contains two subdirectories: `devices` and `drivers`.

In `drivers`, there should be a `nunchuk` subdirectory, but no symbolic link to a device yet. In `devices` you should see some devices, but not the Nunchuk one yet.

Declaring an I2C device

To allow the kernel to manage our Nunchuk device, let's declare the device in the custom Device Tree for our board. The declaration of the I2C1 bus will then look as follows:

```
&i2c1 {
        status = "okay";
        clock-frequency = <100000>;

        nunchuk: joystick@52 {
                compatible = "nintendo,nunchuk";
                reg = <0x52>;
        };
};
```

Here are a few notes:

- The `clock-frequency` property is used to configure the bus to operate at 100 KHz. This is supposed to be required for the Nunchuk.
- The Nunchuk device is added through a child node in the I2C controller node.
- For the kernel to *probe* and drive our device, it's required that the `compatible` string matches one of the `compatible` strings supported by the driver.
- The `reg` property is the address of the device on the I2C bus. If it doesn't match, the driver will probe the device but won't be able to communicate with it.

Recompile your Device Tree and reboot your kernel with the new binary.

You can now load your module again, and this time, you should see that the Nunchuk driver probed the Nunchuk device:

```
# modprobe nunchuk
```

 © 2004-2026 Bootlin, CC BY-SA license

```
[ 66.680455] nunchuk: loading out-of-tree module taints kernel.
[ 66.687645] input: Wii Nunchuk as /devices/platform/ocp/48000000.interconnect/\
    48000000.interconnect:segment@0/4802a000.target-module/4802a000.i2c/i2c-1/1-0052/input/\
    input0
[ 66.695421] Nunchuk device probed successfully
```

List the contents of `/sys/bus/i2c/drivers/nunchuk` once again. You should now see a symbolic link corresponding to our new device.

Also list `/sys/bus/i2c/devices/` again. You should now see the Nunchuk device, which can be recognized through its `0052` address. Follow the link and you should see a symbolic link back to the Nunchuk driver!

We are not ready to use this input device yet, but at least we can test that we get bytes when buttons or the joypad are used. In the below command, use the same number as in the message you got in the console (`event0` for `input0` for example):

```
# cat /dev/input/event0 | od -x
```

Caution: using `od` directly on input event files should work but is currently broken with the Musl library. We are investigating this issue.

We will use the Nunchuk to control audio playback in an upcoming lab.

Setting the board's model name

Modify the custom Device Tree file one last time to override the model name for your system. Set the `model` property to `BeagleBone Black media player`. Don't hesitate to ask your instructor if you're not sure how.

Recompile the device tree, and reboot the board with it. You should see the new model name in two different places:

- In the first kernel messages on the serial console.
- In `/sys/firmware/devicetree/base/model`. This can be handy for a distribution to identify the device it's running on. By the way, you can explore `/sys/firmware/devicetree` and find that every subdirectory corresponds to a DT node, and every file corresponds to a DT property.

Committing kernel tree changes

Now that our changes to the kernel sources are over, create a branch for your changes and create a patch for them. **Please don't skip this step** as we need it for the next labs.

First, if not done yet, you should set your identity and e-mail address in git:

```
git config --global user.email "linus@bootlin.com"
git config --global user.name "Linus Torvalds"
```

This is necessary to create a commit with the `git commit -s` command, as required by the Linux kernel contribution guidelines.

Let's create the branch and the patch now:

```
git checkout -b bootlin-labs
git add arch/arm/boot/dts/ti/omap/am335x-boneblack-custom.dts \
arch/arm/boot/dts/ti/omap/Makefile
git commit -as -m "Custom DTS for Bootlin lab"
```

We can now create the patch:

```
git format-patch stable/linux-6.6.y
```

This should generate a `0001-Custom-DTS-for-Bootlin-lab.patch` file.

Creating the branch will impact the versions of the kernel and the modules. Compile your kernel and install your modules again (not necessary for the Nunchuk one for the moment) and see the version changes through the new base directory for modules.

To save space for the next lab, remove the old directory under `lib/modules` containing the ”dirty” modules.

Don’t forget to update the kernel your board boots.

That’s all for now!

© 2004-2026 Bootlin, CC BY-SA license

Filesystems - Block file systems

Objective: configure and boot an embedded Linux system relying on block storage

After this lab, you will be able to:

- Produce file system images.
- Configure the kernel to use these file systems
- Use the tmpfs file system to store temporary files
- Load the kernel and DTB from a FAT partition

Goals

After doing the *A tiny embedded system* lab, we are going to copy the filesystem contents to the SD card. The storage will be split into several partitions, and your board will boot on an root filesystem on this SD card, without using NFS anymore.

Setup

Throughout this lab, we will continue to use the root filesystem we have created in the `$HOME/embedded-linux-bbb-labs/tinysystem/nfsroot` directory, which we will progressively adapt to use block filesystems.

Filesystem support in the kernel

Recompile your kernel with support for SquashFS and ext4[9].

Boot your board with this new kernel and on the NFS filesystem you used in this previous lab.

Now, check the contents of `/proc/filesystems`. You should see that ext4 and SquashFS are now supported.

Add partitions to the SD card

Plug the SD card in your workstation.

Using `cfdisk /dev/mmcblk0`, add two partitions, starting from the beginning of the remaining space, with the following properties:

- A second primary partition, 8 MB big, for the root filesystem
- A third primary partition, that fills the rest of the SD card, that will be used for the data filesystem

Save and exit when you are done.

Data partition on the SD card

As we are going to use SquashFS to store the root file system, we will no longer be able to write files uploaded through the web interface on it. As an alternative, we are going to create a data partition that will contain these files.

Using the `mkfs.ext4` create a journaled file system on the third partition of the SD card:

```
$ sudo mkfs.ext4 -L data -E nodiscard /dev/mmcblk0p3
```

- `-L` assigns a volume name to the partition

[9] Basic configuration options for these filesystems will be sufficient. No need for things like extended attributes.

- `-E nodiscard` disables bad block discarding. While this should be a useful option for cards with bad blocks, skipping this step saves long minutes in SD cards.

Now, mount this new partition and move the contents of the `/www/upload/files` directory (in your target root filesystem) into it. The goal is to use the data partition of the SD card as the storage for the uploaded images.

Insert the SD card in your board and boot. You should see the partitions in `/proc/partitions`.

Mount this data partition on `/www/upload/files`.

Once this works, modify the startup scripts in your root filesystem to do it automatically at boot time.

Reboot your target system and with the `mount` command, check that `/www/upload/files` is now a mount point for the last SD card partition. Also make sure that you can still upload new images, and that these images are listed in the web interface.

Adding a tmpfs partition for log files

For the moment, the upload script was storing its log file in `/www/upload/files/upload.log`. To avoid seeing this log file in the directory containing uploaded files, let's store it in `/var/log` instead.

Add the `/var/log/` directory to your root filesystem and modify the startup scripts to mount a `tmpfs` filesystem on this directory. You can test your `tmpfs` mount command line on the system before adding it to the startup script, in order to be sure that it works properly.

Modify the `www/cgi-bin/upload.cfg` configuration file to store the log file in `/var/log/upload.log`. You will lose your log file each time you reboot your system, but that's OK in our system. That's what `tmpfs` is for: temporary data that you don't need to keep across system reboots.

Reboot your system and check that it works as expected.

Making a SquashFS image

We are going to store the root filesystem in a SquashFS filesystem in the second partition of the SD card.

In order to create SquashFS images on your host, you need to install the `squashfs-tools` package. Now create a SquashFS image of your NFS root directory.

Finally, using the `dd` command, copy the file system image to the second partition of the SD card.

Booting on the SquashFS partition

In the U-boot shell, configure the kernel command line to use the second partition of the SD card as the root file system. Also add the `rootwait` boot argument, to wait for the SD card to be properly initialized before trying to mount the root filesystem. Since the SD cards are detected asynchronously by the kernel, the kernel might try to mount the root filesystem too early without `rootwait`.

Check that your system still works.

Loading the kernel and DTB from the SD card

In order to let the kernel boot on the board autonomously, we can copy the kernel image and DTB in the boot partition we created previously.

Insert the SD card in your PC, it will get auto-mounted. Copy the kernel and device tree to the `boot` partition.

Insert the SD card back in the board and reset it. You should now be able to load the DTB and kernel image from the SD card and boot with:

```
=> load mmc 0:1 0x81000000 zImage
=> load mmc 0:1 0x82000000 am335x-boneblack-custom.dtb
```

 © 2004-2026 Bootlin, CC BY-SA license

```
=> bootz 0x81000000 - 0x82000000
```

You are now ready to modify `bootcmd` to boot the board from SD card. But first, save the settings for booting from `tftp`:

```
=> setenv bootcmdtftp ${bootcmd}
```

This will be useful to switch back to `tftp` booting mode later in the labs.

Finally, using `editenv bootcmd`, adjust `bootcmd` so that the board starts using the kernel from the SD card.

Now, reset the board to check that it boots in the same way from the SD card.

Now, the whole system (bootloader, kernel and filesystems) is stored on the SD card. That's very useful for product demos, for example. You can switch demos by switching SD cards, and the system depends on nothing else. In particular, no networking is necessary.

© 2004-2026 Bootlin, CC BY-SA license

Third party libraries and applications

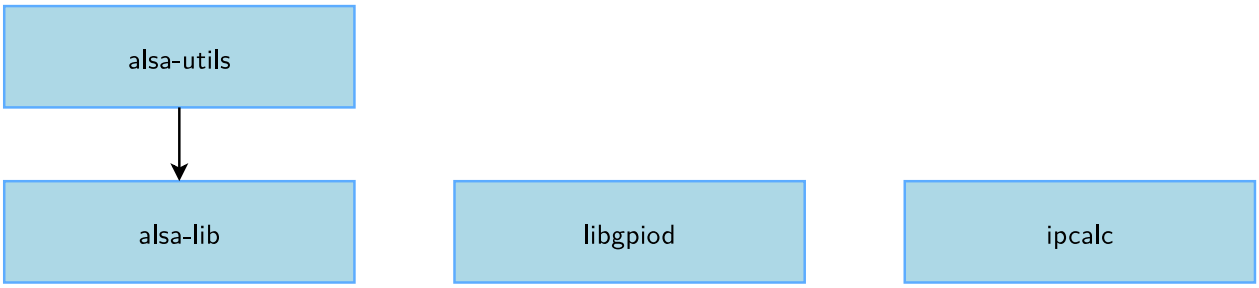
Objective: Learn how to leverage existing libraries and applications: how to configure, compile and install them

To illustrate how to use existing libraries and applications, we will extend the small root filesystem built in the *A tiny embedded system* lab to add the *ALSA* libraries and tools to run basic sound support tests, and the *libgpiod* library and executables to manage GPIOs. *ALSA* stands for *Advanced Linux Sound Architecture*, and is the Linux audio subsystem.

We'll see that manually reusing existing libraries is quite tedious, so that more automated procedures are necessary to make it easier. However, learning how to perform these operations manually will significantly help you when you face issues with more automated tools.

Figuring out library dependencies

We're going to integrate the *alsa-utils*, *libgpiod* and *ipcalc* executables. In our case, the dependency chain for *alsa-utils* is quite simple, it only depends on the *alsa-lib* library. *libgpiod* and *ipcalc* are standalone and don't have any dependency.



Of course, all these libraries rely on the C library, which is not mentioned here, because it is already part of the root filesystem built in the *A tiny embedded system* lab. You might wonder how to figure out this dependency tree by yourself. Basically, there are several ways, that can be combined:

- Read the library documentation, which often mentions the dependencies;
- Read the help message of the `configure script` (by running `./configure --help`).
- By running the `configure` script, compiling and looking at the errors.

To configure, compile and install all the components of our system, we're going to start from the bottom of the tree with *alsa-lib*, then continue with *alsa-utils*. Then, we will also build *libgpiod* and *ipcalc*.

Preparation

For our cross-compilation work, we will need two separate spaces:

- A *staging* space in which we will directly install all the packages: non-stripped versions of the libraries, headers, documentation and other files needed for the compilation. This *staging* space can be quite big, but will not be used on our target, only for compiling libraries or applications;
- A *target* space, in which we will only copy the required files from the *staging* space: binaries and libraries, after stripping, configuration files needed at runtime, etc. This target space will take a lot less space than the *staging* space, and it will contain only the files that are really needed to make the system work on the target.

 © 2004-2026 Bootlin, CC BY-SA license

To sum up, the *staging* space will contain everything that's needed for compilation, while the *target* space will contain only what's needed for execution.

Create the `$HOME/embedded-linux-bbb-labs/thirdparty` directory, and inside, create two directories: `staging` and `target`.

For the target, we need a basic system with BusyBox and initialization scripts. We will reuse the system built in the *A tiny embedded system* lab, so copy this system in the target directory:

```
$ cp -a $HOME/embedded-linux-bbb-labs/tinysystem/nfsroot/* target/
```

Note that for this lab, a lot of typing will be required. To save time typing, we advise you to copy and paste commands from the electronic version of these instructions.

Testing

Make sure the `target/` directory is exported by your NFS server to your board by modifying `/etc/exports` and restarting your NFS server.

Make your board boot from this new directory through NFS.

alsa-lib

alsa-lib is a library supposed to handle the interaction with the ALSA subsystem. It is available at https://alsa-project.org. Download version 1.2.16, and extract it in `$HOME/embedded-linux-bbb-labs/thirdparty/`.

Tip: if the website for any of the source packages that we need to download in the next sections is down, a great mirror that you can use is http://sources.buildroot.net/.

Back to *alsa-lib* sources, look at the `configure` script and see that it has been generated by `autoconf` (the header contains a sentence like *Generated by GNU Autoconf 2.69*). Most of the time, `autoconf` comes with `automake`, that generates Makefiles from `Makefile.am` files. So *alsa-lib* uses a rather common build system. Let's try to configure and build it:

```
$ ./configure
$ make
```

If you look at the generated binaries, you'll see that they are x86 ones because we compiled the sources with gcc, the default compiler. This is obviously not what we want, so let's clean-up the generated objects and tell the `configure` script to use the ARM cross-compiler:

```
$ make clean
$ CC=arm-linux-gcc ./configure
```

Of course, the arm-linux-gcc cross-compiler must be in your `PATH` prior to running the configure script. The `CC` environment variable is the classical name for specifying the compiler to use.

Quickly, you should get an error saying:

```
checking whether we are cross compiling... configure: error: in `/home/tux/
embedded-linux-bbb-labs/thirdparty/alsa-lib-1.2.16':
configure: error: cannot run C compiled programs.
If you meant to cross compile, use `--host'.
See `config.log' for more details
```

If you look at the `config.log` file, you can see that the `configure` script compiles a binary with the cross-compiler and then tries to run it on the development workstation. This is a rather usual thing to do for a `configure` script, and that's why it tests so early that it's actually doable, and bails out if not.

Obviously, it cannot work in our case, and the scripts exits. The job of the `configure` script is to test the configuration of the system. To do so, it tries to compile and run a few sample applications to test if this

library is available, if this compiler option is supported, etc. But in our case, running the test examples is definitely not possible.

We need to tell the `configure` script that we are cross-compiling, and this can be done using the `--build` and `--host` options, as described in the help of the `configure` script:

```
System types:
  --build=BUILD configure for building on BUILD [guessed]
  --host=HOST cross-compile to build programs to run on HOST [BUILD]
```

The `--build` option allows to specify on which system the package is built, while the `--host` option allows to specify on which system the package will run. By default, the value of the `--build` option is guessed and the value of `--host` is the same as the value of the `--build` option. The value is guessed using the `./config.guess` script, which on your system should return `x86_64-pc-linux-gnu`. See https://www.gnu.org/software/autoconf/manual/html_node/Specifying-Names.html for more details on these options.

So, let's override the value of the `--host` option:

```
$ ./configure --host=arm-linux
```

Note that `CC` is not required anymore. It is implied by `--host`.

The `configure` script should end properly now, and create a Makefile.

However, there is one subtle issue to handle. We need to tell *alsa-lib* to disable a feature called *alsa topology*. *alsa-lib* will build fine but we will encounter some problems afterwards, during *alsa-utils* building. So you should configure *alsa-lib* as follows:

```
$ ./configure --host=arm-linux --disable-topology
```

Run the `make` command, which should run just fine.

Look at the result of compiling in `src/.libs`: a set of object files and a set of `libasound.so*` files.

The `libasound.so*` files are a dynamic version of the library. The shared library itself is `libasound.so.2.0.0`, it has been generated by the following command line:

```
$ arm-linux-gcc -shared conf.o confmisc.o input.o output.o async.o error.o dlmisc.o socket.o \
    shmarea.o userfile.o names.o -lm -ldl -lpthread -lrt -Wl,-soname -Wl,libasound.so.2 -o \
    libasound.so.2.0.0
```

And creates the symbolic links `libasound.so` and `libasound.so.2`.

```
$ ln -s libasound.so.2.0.0 libasound.so.2
$ ln -s libasound.so.2.0.0 libasound.so
```

These symlinks are needed for two different reasons:

- `libasound.so` is used at compile time when you want to compile an application that is dynamically linked against the library. To do so, you pass the `-lLIBNAME` option to the compiler, which will look for a file named `lib<LIBNAME>.so`. In our case, the compilation option is `-lasound` and the name of the library file is `libasound.so`. So, the `libasound.so` symlink is needed at compile time;
- `libasound.so.2` is needed because it is the *SONAME* of the library. *SONAME* stands for *Shared Object Name*. It is the name of the library as it will be stored in applications linked against this library. It means that at runtime, the dynamic loader will look for exactly this name when looking for the shared library. So this symbolic link is needed at runtime.

To know what's the *SONAME* of a library, you can use:

 © 2004-2026 Bootlin, CC BY-SA license

```
$ arm-linux-readelf -d libasound.so.2.0.0
```

and look at the (SONAME) line. You'll also see that this library needs the C library, because of the (NEEDED) line on libc.so.0.

The mechanism of SONAME allows to change the library without recompiling the applications linked with this library. Let's say that a security problem is found in the *alsa-lib* release that provides *libasound 2.0.0*, and fixed in the next *alsa-lib* release, which will now provide *libasound 2.0.1*.

You can just recompile the library, install it on your target system, change the libasound.so.2 link so that it points to libasound.so.2.0.1 and restart your applications. And it will work, because your applications don't look specifically for libasound.so.2.0.0 but for the *SONAME* libasound.so.2.

However, it also means that as a library developer, if you break the ABI of the library, you must change the *SONAME*: change from libasound.so.2 to libasound.so.3.

Finally, the last step is to tell the configure script where the library is going to be installed. Most configure scripts consider that the installation prefix is /usr/local/ (so that the library is installed in /usr/local/lib, the headers in /usr/local/include, etc.). But in our system, we simply want the libraries to be installed in the /usr prefix, so let's tell the configure script about this:

```
$ ./configure --host=arm-linux --disable-topology --prefix=/usr
$ make
```

For this library, this option may not change anything to the resulting binaries, but for safety, it is always recommended to make sure that the prefix matches where your library will be running on the target system.

Do not confuse the *prefix* (where the application or library will be running on the target system) from the location where the application or library will be installed on your host while building the root filesystem.

For example, *libasound* will be installed in $HOME/embedded-linux-bbb-labs/thirdparty/target/usr/lib/ because this is the directory where we are building the root filesystem, but once our target system will be running, it will see *libasound* in /usr/lib.

The prefix corresponds to the path in the target system and **never** on the host. So, one should **never** pass a prefix like $HOME/embedded-linux-bbb-labs/thirdparty/target/usr, otherwise at runtime, the application or library may look for files inside this directory on the target system, which obviously doesn't exist! By default, most build systems will install the application or library in the given prefix (/usr or /usr/local), but with most build systems (including *autotools*), the installation prefix can be overridden, and be different from the configuration prefix.

We now only have the installation process left to do.

First, let's make the installation in the *staging* space:

```
$ make DESTDIR=$HOME/embedded-linux-bbb-labs/thirdparty/staging install
```

Now look at what has been installed by *alsa-lib*:

- Some configuration files in /usr/share/alsa
- The headers in /usr/include
- The shared library and its libtool (.la) file in /usr/lib
- A pkgconfig file in /usr/lib/pkgconfig. We'll come back to these later

Finally, let's install the library in the *target* space:

1. Create the target/usr/lib directory, it will contain the stripped version of the library
2. Copy the dynamic version of the library. Only libasound.so.2 and libasound.so.2.0.0 are needed, since libasound.so.2 is the *SONAME* of the library and libasound.so.2.0.0 is the real binary:

- `$ cp -a staging/usr/lib/libasound.so.2* target/usr/lib`

3. Measure the size of the `target/usr/lib/libasound.so.2.0.0` library before stripping.
4. Strip the library:

```
$ arm-linux-strip target/usr/lib/libasound.so.2.0.0
```

5. Measure the size of the `target/usr/lib/libasound.so.2.0.0` library library again after stripping. How many unnecessary bytes were saved?

Then, we need to install the *alsa-lib* configuration files:

```
$ mkdir -p target/usr/share
$ cp -a staging/usr/share/alsa target/usr/share
```

Now, we need to adjust one small detail in one of the configuration files. Indeed, `/usr/share/alsa/alsa.conf` assumes a UNIX group called `audio` exists, which is not the case on our very small system. So edit this file, and replace `defaults.pcm.ipc_gid audio` by `defaults.pcm.ipc_gid 0` instead.

And we're done with *alsa-lib*!

Alsa-utils

Download *alsa-utils* from the ALSA official webpage. We tested the lab with version 1.2.16. The `gettext` package is needed during the build so we have to install it.

```
sudo apt install gettext pkg-config
```

Once uncompressed, we quickly discover that the *alsa-utils* build system is based on the *autotools*, so we will work once again with a regular `configure` script.

As we've seen previously, we will have to provide the prefix and host options:

```
$ ./configure --host=arm-linux --prefix=/usr
```

Now, we should quiclky get an error in the execution of the `configure` script:

```
checking for libasound headers version >= 1.2.13 (1.2.13)... not present.
configure: error: Sufficiently new version of libasound not found.
```

Again, we can check in `config.log` what the `configure` script is trying to do:

```
configure:15855: checking for libasound headers version >= 1.2.13 (1.2.13)
configure:15902: arm-linux-gcc -c -g -O2 conftest.c >&5
conftest.c:24:10: fatal error: alsa/asoundlib.h: No such file or directory
```

Of course, since *alsa-utils* uses *alsa-lib*, it includes its header file! So we need to tell the C compiler where the headers can be found: there are not in the default directory `/usr/include/`, but in the `/usr/include` directory of our *staging* space. The help text of the `configure` script says:

```
  CPPFLAGS    (Objective) C/C++ preprocessor flags, e.g. -I<include dir> if
              you have headers in a nonstandard directory <include dir>
```

Let's use it:

```
$ CPPFLAGS=-I$HOME/embedded-linux-bbb-labs/thirdparty/staging/usr/include \
./configure --host=arm-linux --prefix=/usr
```

Now, it should stop a bit later, this time with the error:

```
checking for snd_ctl_open in -lasound... no
configure: error: No linkable libasound was found.
```

 © 2004-2026 Bootlin, CC BY-SA license

The `configure` script tries to compile an application against *libasound* (as can be seen from the `-lasound` option): *alsa-utils* uses *alsa-lib*, so the `configure` script wants to make sure this library is already installed. Unfortunately, the `ld` linker doesn't find it. So, let's tell the linker where to look for libraries using the `-L` option followed by the directory where our libraries are (in `staging/usr/lib`). This `-L` option can be passed to the linker by using the `LDFLAGS` at configure time, as told by the help text of the `configure` script:

```
  LDFLAGS     linker flags, e.g. -L<lib dir> if you have libraries in a
              nonstandard directory <lib dir>
```

Let's use this `LDFLAGS` variable:

```
$ LDFLAGS=-L$HOME/embedded-linux-bbb-labs/thirdparty/staging/usr/lib \
    CPPFLAGS=-I$HOME/embedded-linux-bbb-labs/thirdparty/staging/usr/include \
    ./configure --host=arm-linux --prefix=/usr
```

Once again, it should fail a bit further down the tests, this time complaining about a missing *curses helper header*. *curses* or *ncurses* is a graphical framework to design UIs in the terminal. This is only used by *alsamixer*, one of the tools provided by *alsa-utils*, that we are not going to use. Hence, we can just disable the build of *alsamixer*.

Of course, if we wanted it, we would have had to build *ncurses* first, just like we built *alsa-lib*.

```
$ LDFLAGS=-L$HOME/embedded-linux-bbb-labs/thirdparty/staging/usr/lib \
    CPPFLAGS=-I$HOME/embedded-linux-bbb-labs/thirdparty/staging/usr/include \
    ./configure --host=arm-linux --prefix=/usr \
    --disable-alsamixer
```

Then, run the compilation with `make`. It should complete successfully.

Let's now begin the installation process. Before really installing in the staging directory, let's install in a dummy directory, to see what's going to be installed (this dummy directory will not be used afterwards, it is only to verify what will be installed before polluting the staging space):

```
$ make DESTDIR=/tmp/alsa-utils/ install
```

The `DESTDIR` variable can be used with all Makefiles based on `automake`. It allows to override the installation directory: instead of being installed in the configuration prefix directory, the files will be installed in `DESTDIR/configuration-prefix`.

Now, let's see what has been installed in `/tmp/alsa-utils/` (run `tree /tmp/alsa-utils`):

```
/tmp/alsa-utils/
|-- lib
|   `-- udev
|       `-- rules.d
|           `-- 90-alsa-restore.rules
|-- usr
|   |-- bin
|   |   |-- aconnect
|   |   |-- alsabat
|   |   |-- alsaloop
|   |   |-- alsaucm
|   |   |-- amidi
|   |   |-- amixer
|   |   |-- aplay
|   |   |-- aplaymidi
|   |   |-- arecord -> aplay
|   |   |-- arecordmidi
```

```
|   |   |-- aseqdump
|   |   |-- aseqnet
|   |   |-- axfer
|   |   |-- iecset
|   |   |-- nhlt-dmic-info
|   |   `-- speaker-test
|   |-- sbin
|   |   |-- alsabat-test.sh
|   |   |-- alsaconf
|   |   |-- alsactl
|   |   `-- alsa-info.sh
|   `-- share
|       |-- alsa
|       |   `-- init
|       |       |-- 00main
|       |       |-- ca0106
|       |       |-- default
|       |       |-- hda
|       |       |-- help
|       |       |-- info
|       |       `-- test
|       |-- locale
|       |   |-- de
|       |   |   `-- LC_MESSAGES
|       |   |       `-- alsa-utils.mo
|       |   |-- eu
|       |   |   `-- LC_MESSAGES
|       |   |       `-- alsa-utils.mo
|       |   |-- fr
|       |   |   `-- LC_MESSAGES
|       |   |       `-- alsa-utils.mo
|       |   |-- ja
|       |   |   `-- LC_MESSAGES
|       |   |       |-- alsaconf.mo
|       |   |       `-- alsa-utils.mo
|       |   |-- ka
|       |   |   `-- LC_MESSAGES
|       |   |       |-- alsaconf.mo
|       |   |       `-- alsa-utils.mo
|       |   |-- ko
|       |   |   `-- LC_MESSAGES
|       |   |       `-- alsa-utils.mo
|       |   |-- ru
|       |   |   `-- LC_MESSAGES
|       |   |       `-- alsaconf.mo
|       |   `-- sk
|       |       `-- LC_MESSAGES
|       |           `-- alsa-utils.mo
|       |-- man
|       |   |-- fr
|       |   |   `-- man8
|       |   |       `-- alsaconf.8
|       |   |-- man1
|       |   |   |-- aconnect.1
```


© 2004-2026 Bootlin, CC BY-SA license

```
|       |   |   |-- alsabat.1
|       |   |   |-- alsactl.1
|       |   |   |-- alsa-info.sh.1
|       |   |   |-- alsaloop.1
|       |   |   |-- amidi.1
|       |   |   |-- amixer.1
|       |   |   |-- aplay.1
|       |   |   |-- aplaymidi.1
|       |   |   |-- arecord.1 -> aplay.1
|       |   |   |-- arecordmidi.1
|       |   |   |-- aseqdump.1
|       |   |   |-- aseqnet.1
|       |   |   |-- axfer.1
|       |   |   |-- axfer-list.1
|       |   |   |-- axfer-transfer.1
|       |   |   |-- iecset.1
|       |   |   |-- nhlt-dmic-info.1
|       |   |   `-- speaker-test.1
|       |   |-- man7
|       |   `-- man8
|       |       `-- alsaconf.8
|       `-- sounds
|           `-- alsa
|               |-- Front_Center.wav
|               |-- Front_Left.wav
|               |-- Front_Right.wav
|               |-- Noise.wav
|               |-- Rear_Center.wav
|               |-- Rear_Left.wav
|               |-- Rear_Right.wav
|               |-- Side_Left.wav
|               `-- Side_Right.wav
`-- var
    `-- lib
        `-- alsa

37 directories, 68 files
```

So, we have:

- The *udev* rules in `lib/udev`
- The *alsa-utils* binaries in `/usr/bin` and `/usr/sbin`
- Some sound samples in `/usr/share/sounds`
- The various translations in `/usr/share/locale`
- The manual pages in `/usr/share/man/`, explaining how to use the various tools
- Some configuration samples in `/usr/share/alsa`.

Now, let's make the installation in the *staging* space:

```
$ make DESTDIR=$HOME/embedded-linux-bbb-labs/thirdparty/staging/ install
```

Then, let's manually install only the necessary files in the *target* space. We are only interested in `speaker-test`:

```
$ cd ..
$ cp -a staging/usr/bin/speaker-test target/usr/bin/
$ arm-linux-strip target/usr/bin/speaker-test
```

And we're finally done with *alsa-utils*!

Now test that all is working fine by running the `speaker-test` util on your board, with the headset provided by your instructor plugged in. You may need to add the missing libraries from the toolchain install directory.

Now you can use:

- `speaker-test` with no arguments to generate *pink noise*
- `speaker-test -t sine` to generate a *sine wave*, optionally with `-f <freq>` for a specific frequency

There you are: you built and ran your first program depending on a library different from the C library.

libgpiod

Compiling libgpiod

We are now going to use *libgpiod* (instead of the deprecated interface in `/sys/class/gpio`, whose executables (`gpiodetect`, `gpioset`, `gpioget`...) will allow us to drive and manage GPIOs from shell scripts.

Here, we will be using the 2.2.x version of *libgpiod*.

```
git clone https://git.kernel.org/pub/scm/libs/libgpiod/libgpiod.git
cd libgpiod
git checkout v2.2.x
```

As we are not starting from a release, we will need to install further development tools to generate some files like the `configure` script:

```
sudo apt install autoconf-archive
```

Now let's generate the files which are present in a release:

```
./autogen.sh
```

Run `./configure --help` script, and see that this script provides a `--enable-tools` option which allows to build the userspace executables that we want.

As this project doesn't have any external library dependency, let's configure *libgpiod* in a similar way as *alsa-utils*:

```
$ ./configure --host=arm-linux --prefix=/usr --enable-tools
```

Now, compile the software:

```
$ make
```

Installation to the *staging* space can be done using the classical `DESTDIR` mechanism:

```
$ make DESTDIR=$HOME/embedded-linux-bbb-labs/thirdparty/staging/ install
```

And finally, only manually install and strip the files needed at runtime in the *target* space:

```
$ cd ..
$ cp -a staging/usr/lib/libgpiod.so.3* target/usr/lib/
$ arm-linux-strip target/usr/lib/libgpiod*
$ cp -a staging/usr/bin/gpio* target/usr/bin/
```

 © 2004-2026 Bootlin, CC BY-SA license

```
$ arm-linux-strip target/usr/bin/gpio*
```

Testing libgpiod

First, connect `GPIO1_28` (pin 12 of connector P9) connected to ground (pin 1 of connector P9), as in the *Accessing Hardware Devices* lab.

Now, let's run the `gpiodetect` command on the target, and check that you can list the various GPIO banks on your system.

```
# gpiodetect
gpiochip0 [gpio-0-31] (32 lines)
gpiochip1 [gpio-32-63] (32 lines)
gpiochip2 [gpio-64-95] (32 lines)
gpiochip3 [gpio-96-127] (32 lines)
```

So, `GPIO1_28` should be part of `gpiochip0`.

We can then get details on GPIOE GPIOs by running `gpioinfo gpiochip0` or on all GPIOs by simply running `gpioinfo`.

You can now read the state of your GPIO:

```
# gpioget -c gpiochip0 28
"28"=inactive
```

Now, connect your wire to 3V3 (pin 3 or 4 of connector P9). You should now read:

```
# gpioget -c gpiochip0 28
"28"=active
```

You see that you didn't have to configure the GPIO as input. *libgpiod* did that for you.

If you have an LED and a small breadboard and appropriate breadboard wires, you could also try to drive the GPIO in output mode. Connect the short pin of the LED to GND, and the long one to the GPIO. Then then following command should light up the diode:

```
gpioset -c gpiochip0 -t0 28=1
```

The `-t0` option makes `gpioset` terminate by itself immediately instead of waiting for the user to interrupt it.

Here's how to turn the LED off:

```
gpioset -c gpiochip0 -t0 28=0
```

`gpioset` offers many more options. Run `gpioset -h` to check by yourself.

ipcalc

After practicing with autotools based packages, let's build *ipcalc*, which is using *Meson* as build system. We won't really need this utility in our system, but at least it has no dependencies and therefore offers an easy way to build our first *Meson* based package.

So, first install the `meson` package:

```
$ sudo apt install meson
```

In the main lab directory, then let's check out the sources through `git`:

```
$ git clone https://gitlab.com/ipcalc/ipcalc.git
$ cd ipcalc/
$ git checkout 1.0.3
```

To cross-compile with *Meson*, we need to create a *cross file*. Let's create the `../cross-file.txt` file with the below contents:

```
[binaries]
c = 'arm-linux-gcc'

[host_machine]
system = 'linux'
cpu_family = 'arm'
cpu = 'cortex-a8'
endian = 'little'
```

We also need to create a special directory for building:

```
$ mkdir cross-build
$ cd cross-build
```

We can now have `meson` create the Ninja build files for us:

```
$ meson --cross-file ../../cross-file.txt --prefix /usr ..
```

We are now ready to build *ipcalc*:

```
$ ninja
```

And now install `ipcalc` to the build space:

```
$ DESTDIR=$HOME/embedded-linux-bbb-labs/thirdparty/staging ninja install
```

Check that the `staging/usr/bin/ipcalc` file is indeed an ARM executable.

The last thing to do is to copy it to the target space and strip it:

```
$ cd ../..
$ cp staging/usr/bin/ipcalc target/usr/bin/
$ arm-linux-strip target/usr/bin/ipcalc
```

Note that we could have asked `ninja install` to strip the executable for us when installing it into the staging directory. To do, this, we would have added a `strip` entry in the cross file, and passed `--strip` to *Meson*. However, it's better to keep files unstripped in the staging space, in case we need to debug them.

You can now test that `ipcalc` works on the target:

```
# ipcalc 192.168.0.100
Address: 192.168.0.100
Address space: Private Use
```

Final touch

To finish this lab completely, and to be consistent with what we've done before, let's strip the C library and its loader too.

First, check the initial size of the binaries:

```
$ ls -l target/lib
```

 © 2004-2026 Bootlin, CC BY-SA license

Then strip the binaries in `/lib`:

```
$ chmod +w target/lib/*.so.*
$ arm-linux-strip target/lib/*.so.*
```

And check the final size:

```
$ ls -l target/lib/
```

© 2004-2026 Bootlin, CC BY-SA license

Using a build system, example with Buildroot

Objectives: discover how a build system is used and how it works, with the example of the Buildroot build system. Build a full Linux system, including the Linux kernel.

Goals

Compared to the previous lab, we are going to build a more elaborate system, still containing *alsa-utils* (and of course its *alsa-lib* dependency), but this time using Buildroot, an automated build system.

The automated build system will also allow us to add more packages and play real audio on our system, thanks to the *Music Player Daemon (mpd)* (https://www.musicpd.org/ and its *mpc* client.

As in a real project, we will also build the Linux kernel from Buildroot, and install the kernel modules in the root filesystem.

Setup

Go to the `$HOME/embedded-linux-bbb-labs/buildroot` directory.

Get Buildroot and explore the source code

The official Buildroot website is available at https://buildroot.org/. Clone the *Git* repository:

```
git clone https://gitlab.com/buildroot.org/buildroot.git
cd buildroot
```

Now checkout the tag corresponding to the latest 2026.02.<n> release (Long Term Support), which we have tested for this lab.

Several subdirectories or files are visible, the most important ones are:

- `boot` contains the Makefiles and configuration items related to the compilation of common bootloaders (GRUB, U-Boot, Barebox, etc.)
- `board` contains board specific configurations and root filesystem overlays.
- `configs` contains a set of predefined configurations, similar to the concept of defconfig in the kernel.
- `docs` contains the documentation for Buildroot.
- `fs` contains the code used to generate the various root filesystem image formats
- `linux` contains the Makefile and configuration items related to the compilation of the Linux kernel
- `Makefile` is the main Makefile that we will use to use Buildroot: everything works through Makefiles in Buildroot;
- `package` is a directory that contains all the Makefiles, patches and configuration items to compile the user space applications and libraries of your embedded Linux system. Have a look at various subdirectories and see what they contain;
- `system` contains the root filesystem skeleton and the *device tables* used when a static `/dev` is used;

 © 2004-2026 Bootlin, CC BY-SA license

- `toolchain` contains the Makefiles, patches and configuration items to generate the cross-compiling toolchain.

Board specific configuration

As we will want Buildroot to build a kernel with a custom configuration, and our custom patch, so let's add our own subdirectory under `board`:

```
mkdir -p board/bootlin/training
```

Then, copy your kernel configuration and kernel patch:

```
cp ../../kernel/linux/.config board/bootlin/training/linux.config
cp ../../kernel/linux/0001-Custom-DTS-for-Bootlin-lab.patch \
  board/bootlin/training/
```

We will configure Buildroot to use this kernel configuration.

Configure Buildroot

In our case, we would like to:

- Generate an embedded Linux system for ARM;
- Use an already existing external toolchain instead of having Buildroot generating one for us;
- Compile the Linux kernel and deploy its modules in the root filesystem;
- Integrate *BusyBox*, *alsa-utils*, *mpd*, *mpc* and *evtest* in our embedded Linux system;
- Integrate the target filesystem into a tarball

To run the configuration utility of Buildroot, simply run:

```
$ make menuconfig
```

Set the following options. Don't hesitate to press the `Help` button whenever you need more details about a given option:

- `Target options`
  - `Target Architecture: ARM (little endian)`
  - `Target Architecture Variant: cortex-A8`
  - `Target ABI: EABIhf`
  - `Floating point strategy: VFPv3-D16`
- `Toolchain`
  - `Toolchain type: External toolchain`
  - `Toolchain: Custom toolchain`
  - `Toolchain path:` use the toolchain you built: `/home/<user>/x-tools/arm-training-linux-musleabihf` (replace `<user>` by your actual user name)
  - `External toolchain gcc version: 15.x`
  - `External toolchain kernel headers series`: Set it to match the version of the kernel headers chosen when building the toolchain.
  - `External toolchain C library: musl (experimental)`

© 2004-2026 Bootlin, CC BY-SA license

  - We must tell Buildroot about our toolchain configuration, so select `Toolchain has SSP support?` and `Toolchain has C++ support?`. Buildroot will check these parameters anyway.

- `Kernel`
  - Enable `Linux Kernel`
  - Set `Kernel version` to `Custom version`
  - Set `Kernel version` to your kernel version. You can use `make kernelversion` to get it from the Linux kernel source tree.
  - Set `Custom kernel patches` to `board/bootlin/training/0001-Custom-DTS-for-Bootlin-lab.patch`
  - Set `Kernel configuration` to `Using a custom (def)config file`)
  - Set `Configuration file path` to `board/bootlin/training/linux.config`
  - Select `Build a Device Tree Blob (DTB)`
  - Set `In-tree Device Tree Source file names` to `ti/omap/am335x-boneblack-custom`
- `Target packages`
  - Keep `BusyBox` (default version) and keep the BusyBox configuration proposed by Buildroot;
  - `Audio and video applications`
    - Select `alsa-utils`, and in the submenu:
      - Only keep `speaker-test`
    - Select `mpd`, and in the submenu:
      - Keep only `alsa`, `vorbis` and `tcp sockets`
    - Select `mpd-mpc`.
  - `Hardware handling`
    - Select `evtest`
      This userspace application allows to test events from input devices. This way, we will be able to test the Nunchuk by getting details about which buttons were pressed.
- `Filesystem images`
  - Select `tar the root filesystem`

Exit the menuconfig interface. Your configuration has now been saved to the `.config` file.

Generate the embedded Linux system

Just run:

```
$ make
```

Buildroot will first create a small environment with the external toolchain, then download, extract, configure, compile and install each component of the embedded system.

All the compilation has taken place in the `output/` subdirectory. Let's explore its contents:

- `build`, is the directory in which each component built by Buildroot is extracted, and where the build actually takes place
- `host`, is the directory where Buildroot installs some components for the host. As Buildroot doesn't want to depend on too many things installed in the developer machines, it installs some tools needed to compile the packages for the target. In our case it installed *pkg-config* (since the version of the host may be ancient) and tools to generate the root filesystem image (*genext2fs*, *makedevs*, *fakeroot*).

 © 2004-2026 Bootlin, CC BY-SA license

- `images`, which contains the final images produced by Buildroot. In our case it contains a tarball of the filesystem, called `rootfs.tar`, plus the compressed kernel and Device Tree binary. Depending on the configuration, there could also a bootloader binary or a full SD card image.
- `staging`, which contains the "build" space of the target system. All the target libraries, with headers and documentation. It also contains the system headers and the C library, which in our case have been copied from the cross-compiling toolchain.
- `target`, is the target root filesystem. All applications and libraries, usually stripped, are installed in this directory. However, it cannot be used directly as the root filesystem, as all the device files are missing: it is not possible to create them without being root, and Buildroot has a policy of not running anything as root.

Run the generated system

Go back to the `$HOME/embedded-linux-bbb-labs/buildroot/` directory. Create a new `nfsroot` directory that is going to hold our system, exported over NFS. Go into this directory, and untar the rootfs using:

```
$ tar xvf ../buildroot/output/images/rootfs.tar
```

Add our `nfsroot` directory to the list of directories exported by NFS in `/etc/exports`.

Also update the kernel and Device Tree binaries used by your board, from the ones compiled by Buildroot in `output/images/`.

Boot the board, and log in (`root` account, no password).

You should now reach a shell.

Loading the USB audio module

You can check that no kernel module is loaded yet. Try to load the `snd_usb_audio` module from the command line.

This should work. Check that Buildroot has deployed the modules for your kernel in `/lib/modules`.

Let's automate this now!

Look at the `/etc/inittab` file generated by Buildroot (ask your instructor if you have any questions), and at the contents of the `/etc/init.d/` directory, in particular of the `rcS` file.

You can see that `rcS` executes or sources all the `/etc/init.d/S??*` files. We can add our own which will load the toplevel modules that we need.

Let's do this by creating an *overlay directory*, typically under our board specific directory, that Buildroot will add after building the root filesystem:

```
mkdir -p board/bootlin/training/rootfs-overlay/
```

Then add a custom startup script, by adding an `etc/init.d/S03modprobe` executable file to the overlay directory, with the below contents:

```
#!/bin/sh
modprobe snd-usb-audio
```

Then, go back to Buildroot's configuration interface:

- `System configuration`
  - Set `Root filesystem overlay directories` to `board/bootlin/training/rootfs-overlay`

Build your image again. This should be quick as Buildroot doesn't need to recompile anything. It will just apply the root filesystem overlay.

© 2004-2026 Bootlin, CC BY-SA license

Update your `nfsroot` directory, reboot the board and check that the `snd_usb_audio` module is loaded as expected.

You can run `speaker-test` to check that audio indeed works.

Testing music playback with mpd and mpc

The next thing we want to do is play real sound samples with the *Music Player Daemon (MPD)*. So, let's add music files [10] for MPD to play:

```
mkdir -p board/bootlin/training/rootfs-overlay/var/lib/mpd/music
cp ../data/music/* board/bootlin/training/rootfs-overlay/var/lib/mpd/music
```

Update your root filesystem. Thanks to NFS, you don't need to restart your system.

Using the `ps` command, check that the `mpd` server was started by the system, as implemented by the `/etc/init.d/S95mpd` script.

If that's the case, you are now ready to run `mpc` client commands to control music playback. First, let's make `mpd` process the newly added music files. Run this command on the target:

```
# mpc update
```

You should see the files getting indexed, by displaying the contents of the `/var/log/mpd.log` file:

```
Jan 01 00:04 : exception: Failed to open '/var/lib/mpd/state': No such file or directory
Jan 01 00:15 : update: added /2-arpent.ogg
Jan 01 00:15 : update: added /6-le-baguette.ogg
Jan 01 00:15 : update: added /4-land-of-pirates.ogg
Jan 01 00:15 : update: added /3-chronos.ogg
Jan 01 00:15 : update: added /1-sample.ogg
Jan 01 00:15 : update: added /7-fireworks.ogg
Jan 01 00:15 : update: added /5-ukulele-song.ogg
```

You can also check the list of available files:

```
# mpc listall
1-sample.ogg
2-arpent.ogg
5-ukulele-song.ogg
3-chronos.ogg
7-fireworks.ogg
6-le-baguette.ogg
4-land-of-pirates.ogg
```

To play files, you first need to create a playlist. Let's create a playlist by adding all music files to it:

```
# mpc add /
```

You should now be able to start playing the songs in the playlist:

```
# mpc play
```

Here are a few further commands for controlling playback:

- `mpc toggle`: toggle between pause and playback modes.
- `mpc next`: switch to the next song in the playlist.

[10] For the most part, these are public domain music files, except a small sample file... See the `README.txt` file in the directory containing the files.

 © 2004-2026 Bootlin, CC BY-SA license

- `mpc prev`: switch to the previous song in the playlist.
- `mpc volume +5`: increase the volume by 5%
- `mpc volume -5`: reduce the volume by 5%

The volume control commands won't work right away. You probably noticed the following error logs when MPD started:

```
Starting mpd: Jan 01 00:00 : server_socket: bind to '0.0.0.0:6600' failed (continuing anyway,
because binding to '[::]:6600' succeeded): Failed to bind socket: Address
in use
Jan 01 00:00 : exception: Failed to open '/var/lib/mpd/database': No such file or directory
Jan 01 00:00 : output: No 'audio_output' defined in config file
Jan 01 00:00 : output: Successfully detected a alsa audio device
```

These are due to invalid configuration options. We will add a custom configuration for MPD, as the standard one provided by Buildroot doesn't perfectly fit our use case. We will simply add this file to our overlay:

```
cp ../data/mpd.conf board/bootlin/training/rootfs-overlay/etc/
```

Run Buildroot again and update your root filesystem. Here again, you don't need to reboot. It's sufficient to restart MPD to make it read the new configuration file:

```
# /etc/init.d/S95mpd restart
```

You can now make sure that modifying the volume works.

Later, we will compile and debug a custom MPD client application.

Analyzing dependencies

It's always useful to understand the dependencies drawn by the packages we build.

First we need to install a *Graphviz*:

```
$ sudo apt install graphviz
```

Now, let's use Buildroot's target to generate a dependency graph:

```
$ make graph-depends
```

We can now study the dependency graph:

```
$ evince output/graphs/graph-depends.pdf
```

In particular, you can see that adding MPD and its client required to compile *Meson* for the host, and in turn, *Python 3* for the host too. This substantially contributed to the build time.

Adding a Buildroot package

We would also like to build our Nunchuk external module with Buildroot. Fortunately, Buildroot has a `kernel-module` infrastructure to build kernel modules.

First, create a `nunchuk-driver` subdirectory under `package` in Buildroot sources.

The first thing is to create a `package/nunchuk-driver/Config.in` file for Buildroot's configuration:

```
config BR2_PACKAGE_NUNCHUK_DRIVER
        bool "nunchuk driver"
        depends on BR2_LINUX_KERNEL
        help
                Linux Kernel module for the I2C Nunchuk.
```

Then add a line to `package/Config.in` to include this file, for example right before the line including `package/nvidia-driver/Config.in`, so that the alphabetic order of configuration options is kept.

Then, the next and last thing you need to do is create `package/nunchuk-driver/nunchuk-driver.mk` describing how to build the package:

```
NUNCHUK_DRIVER_VERSION = 1.0
NUNCHUK_DRIVER_SITE = $(HOME)/embedded-linux-bbb-labs/hardware/data/nunchuk
NUNCHUK_DRIVER_SITE_METHOD = local
NUNCHUK_DRIVER_LICENSE = GPL-2.0

$(eval $(kernel-module))
$(eval $(generic-package))
```

Then, configure Buildroot to build your package, run Buildroot and update your root filesystem.

Can you load the `nunchuk` module now? If everything's fine, add a line to `/etc/init.d/S03modprobe` for this driver, and update your root filesystem once again.

Testing the Nunchuk

Now that we have the `nunchuk` driver loaded and that Buildroot compiled `evtest` for the target, thanks to Buildroot, we can now test the input events coming from the Nunchuk.

```
# evtest
No device specified, trying to scan all of /dev/input/event*
Available devices:
/dev/input/event0: pmic_onkey
/dev/input/event1: Logitech Inc. Logitech USB Headset H340 Consumer Control
/dev/input/event2: Logitech Inc. Logitech USB Headset H340
/dev/input/event3: Wii Nunchuk
Select the device event number [0-3]:
```

Enter the number corresponding to the Nunchuk device.

You can now press the Nunchuk buttons, use the joypad, and see which input events are emitted.

By the way, you can also test which input events are exposed by the driver for your audio headset (if any), which doesn't mean that they physically exist.

Commit your changes

As we are going to reuse our Buildroot changes in the next labs, let's commit them into a branch:

```
git checkout -b bootlin-labs
git add board/bootlin/ package/nunchuk-driver/ package/Config.in
git commit -as -m "Bootlin lab changes"
```

Going further

If you finish your lab before the others

- For more music playing fun, you can install the `ario` or *cantata* MPD client on your host machine (`sudo apt install ario`, `sudo apt install cantata`), configure it to connect to the IP address of your target system with the default port, and you will also be able to control playback from your host machine.

 © 2004-2026 Bootlin, CC BY-SA license

System Integration - Using systemd

Objectives: Get familiar with the systemd *init system.*

Goals

Compared to the previous lab, we go on increasing the complexity of the system, this time by using the *systemd* init system, and by taking advantage of it to add a few extra features, in particular ones that will be useful for debugging in the next lab.

Setup

Since *systemd* requires the GNU C library, we are going to make a new Buildroot build in a new working directory, and using a different cross-compiling toolchain.

So, create and enter the `$HOME/embedded-linux-bbb-labs/integration` directory.

Make a new clone of Buildroot from the existing local Git repository, and checkout our `bootlin-labs` branch:

```
git clone $HOME/embedded-linux-bbb-labs/buildroot/buildroot
cd buildroot
git checkout bootlin-labs
```

Root filesystem overlay

Remove `etc/init.d/` from the root filesystem overlay. It was adapted to *BusyBox init*, not to *systemd*:

```
rm -r board/bootlin/training/rootfs-overlay/etc/init.d/
```

Buildroot configuration

Configure Buildroot as follows:

- `Target options`
  - Select the same architecture and CPU settings as in the previous lab.
- `Toolchain`
  - `Toolchain type`: `External toolchain`
  - `Toolchain`: `Bootlin toolchains`
    This time, we will use a Bootlin ready-made toolchain for *glibc*, as this is necessary for using *systemd*.
  - `Toolchain origin`: `Toolchain to be downloaded and installed`
  - `Bootlin toolchain variant`: `armv7-eabihf glibc bleeding-edge 2025.08-1`
  - Select `Copy gdb server to the Target`
- `System configuration`
  - `Init system`: `systemd`
  - `Root filesystem overlay directories`: `board/bootlin/training/rootfs-overlay`
- `Kernel`

  - Enable `Linux Kernel`
  - Set `Kernel version` to `Custom version`
  - Set `Kernel version` to your kernel version. You can use `make kernelversion` to get it from the Linux kernel source tree.
  - Set `Custom kernel patches` to `board/bootlin/training/0001-Custom-DTS-for-Bootlin-lab.patch`
  - Set `Kernel configuration` to `Using a custom (def)config file`)
  - Set `Configuration file path` to `board/bootlin/training/linux.config`
  - Select `Build a Device Tree Blob (DTB)`
  - Set `In-tree Device Tree Source file names` to `ti/omap/am335x-boneblack-custom`
- `Target packages`
  - `Audio and video applications`
    - We won't need `alsa-utils` this time.
    - Select `mpd`, and in the submenu:
      - Keep only `alsa`, `vorbis` and `tcp sockets`
    - Select `mpd-mpc`.
  - `Hardware handling`
    - Select `nunchuk driver`
  - `Networking applications`
    - Select `dropbear`, a lightweight SSH server used instead of OpenSSH in most embedded devices. You don't need to enable client support (building an SSH client).
- `Filesystem images`
  - Select `tar the root filesystem`

Build and test the new system

Now build the full system.

Once the build is over, generate the dependency graph again and find out the new dependencies introduced by using *systemd*.

To test the new system, create a new `nfsroot` directory, extract the new root filesystem into it, and boot your board on it through NFS.

You should see the system booting through *systemd*, with all the *systemd* targets and system services starting one by one, with a total boot time which looks slower than before. That's because the system configuration is more complex, but also more versatile, being ready to run more complex services and applications.

You can ask *systemd* to show you the various services which were started:

```
# systemctl status
```

You can also check all the mounted filesystems and be impressed:

```
# mount
```

 © 2004-2026 Bootlin, CC BY-SA license

Inspecting the system

On the target, look at the contents of `/lib/systemd`. You will see the implementation of most *systemd* targets and services.

In particular, check out `/lib/systemd/user/` containing some unnecessary targets in our case such as `bluetooth.target`.

However, check the `mpd.service` file for our MPD server. This should help you to realize all the options provided by *systemd* to start and control system services, while keeping the system secure and their resources under control.

You won't be able to match this level of control and security in a "hand-made" system.

Note: you may notice that a "systemd-network-generator" unit fails to start. It is due to systemd failing to parse correctly the `ip` *parameter from the kernel commandline. You can circumvent this issue by setting the* `autoconf` *field (currently not set at all) of the* `ip` *parameter to* `none`*. You can refer to nfsroot documentation to learn more about this option.*

Understanding automatic module loading with Udev

Check the currently loaded modules on your system. Surprise: both the Nunchuk and USB audio modules are already loaded. We didn't have anything to set up and *systemd* automatically loaded the modules associated to connected hardware.

Let's find out why...

On the target, go to `/lib/udev/rules.d`. You will find all the standard rules for *Udev*, the part of *systemd* which handles hardware events, takes care of the permissions and ownership of device files, notifies other userspace programs, and among others, load kernel modules.

Open `80-drivers.rules`, which is the rule allowing *Udev* to load kernel modules for detected devices. Here is its most important line:

```
ENV{MODALIAS}=="?*", RUN{builtin}+="kmod load '$env{MODALIAS}'"
```

This is when the `modules.alias` file comes into play. When a new device is found, the kernel passes a `MODALIAS` environment variable to *Udev*, containing which bus this happened on and the attributes of the device on this bus. Thanks to the module aliases, the right module gets loaded. We already explained that in the lectures when talking about the output of `make modules_install`.

Find where the `modules.alias` file is located and you will find the two lines that allowed to load our `snd_usb_audio` and `nunchuk` modules:

```
...
alias usb:v*p*d*dc*dsc*dp*ic01isc01ip*in* snd_usb_audio
alias usb:v2B53p0031d*dc*dsc*dp*ic*isc*ip*in* snd_usb_audio
...
alias of:N*T*Cnintendo,nunchuk nunchuk
```

For `snd_usb_audio`, there are many possible matching values, so it's not straightforward to be sure which matched your particular device.

However, you can find in *sysfs* which `MODALIAS` was emitted for your device:

```
# cd /sys/class/sound/card0/device
# ls -la
# cat modalias
usb:v1B3Fp2008d0100dc00dsc00dp00ic01isc01ip00in00
```

With a bit of patience, you could find the matching line in the `modules.alias` file.

If you want to see the information sent to *Udev* by the kernel when a new device is plugged in, here are a few debugging commands.

First unplug your device and run:

```
# udevadm monitor
```

Then plug in your headset again. You will find all the events emitted by the kernel, and with the same string (with `UDEV` instead of `KERNEL`), the time when *Udev* finished processing each event.

You can also see the `MODALIAS` values carried by these events:

```
# udevadm monitor --env
```

As far as the Nunchuk is concerned, we cannot easily remove it from the Device Tree and add it back, but it's easier to find its `MODALIAS` value:

```
# cd /sys/bus/i2c/devices
# ls -la
```

Here you will recognize our Nunchuk device through its `0x52` address.

```
# cd X-0052
# ls -la
# cat modalias
of:NjoystickT(null)Cnintendo,nunchuk
```

Here the bus is `of`, meaning *Open Firmware*, which was the former name of the Device Tree. When an event was emitted by the kernel with this `MODALIAS` string, the `nunchuk` module got loaded by *Udev* thanks to the matching alias.

This actually happened when *systemd* ran the *coldplugging* operation: at system startup, it asked the kernel to emit hotplug events for devices already present when the system booted:

```
[  OK  ] Finished Coldplug All udev Devices.
```

On non-x86 platforms, that's typically for devices described in the Device Tree. This way, both *static* and *hotplugged* devices can be handled in the same way, using the same *Udev* rules.

Testing your system

Make sure that audio playback still works on your system:

```
# mpc update
# mpc add /
# mpc play
```

If it doesn't, look at the *systemd* logs in your serial console history. *systemd* should let you know about the failing services and the commands to run to get more details.

 © 2004-2026 Bootlin, CC BY-SA license

Application development and application debugging

Objective: compile an application against a Buildroot build space and debug it remotely.

Setup

We will continue to use the same root filesystem.

Our goal is to compile and debug our own *MPD* client. This client will be driven by the Nunckuk to switch between audio tracks, and to adjust the playback volume.

However, this client will be used together with `mpc`, as it won't be able to create the playlist and start the playback. It will just be used to control the volume and switch between songs. So, you need to run `mpc` commands first before trying the new client:

```
mpc update
mpc add /
mpc pause
```

We will use the new client to resume playback.

Compile your own application

Go to the `$HOME/embedded-linux-bbb-labs/appdev` directory.

In the lab directory the file `nunchuk-mpd-client.c` contains an application which implements a simple MPD client based on the *libmpdclient* library. As *mpc* is also based on this library, Buildroot already compiled it and added it to our root filesystem. What's special in this application is that it allows to drive music playback through our Nunchuk.

Buildroot has generated toolchain wrappers in `output/host/bin`, which make it easier to use the toolchain, since these wrappers pass some mandatory flags (especially the `--sysroot` *gcc* flag, which tells *gcc* where to look for the headers and libraries). This way, we can compile our application outside of Buildroot, as often as we want.

Let's add this directory to our `PATH`:

```
$ export PATH=$HOME/embedded-linux-bbb-labs/integration/buildroot/output/host/bin:$PATH
```

Let's try to compile the application:

```
$ arm-linux-gcc -o nunchuk-mpd-client nunchuk-mpd-client.c
```

The compiler complains about undefined references to some symbols in *libmpdclient*. This is normal, since we didn't tell the compiler to link with this library. So let's use `pkg-config` to query the *pkg-config* database about the list of libraries needed to build an application against *libmpdclient*[11]:

[11] Normally, `output/host/bin` has a special `pkg-config` that automatically knows where to look, so it already knows the right paths to find `.pc` files and their sysroot, but here there is an open issue with Buildroot 2024.02 which forced us to set `PKG_CONFIG_PATH` to make it point to where the `.pc` files are found.

```
$ export PKG_CONFIG_PATH=$HOME/embedded-linux-bbb-labs/integration/buildroot/output/host/\
    arm-buildroot-linux-gnueabihf/sysroot/usr/lib/pkgconfig
$ arm-linux-gcc -o nunchuk-mpd-client nunchuk-mpd-client.c \
$(pkg-config --libs libmpdclient)
```

Copy the `nunchuk-mpd-client` executable to the `/root` directory of the root filesystem, and then strip it.

Back to target system, try to run the program:

```
# /root/nunchuk-mpd-client
ERROR: didn't manage to find the Nunchuk device in /dev/input. Is the Nunchuk driver loaded?
```

Enable debugging tools

In order to debug our application, let's make Buildroot build some debugging tools for our root filesystem. This is also an opportunity to enable `perf`, that we are using later on during this lab. Go back to the Buildroot configuration interface and enable the following options:

- `Kernel`
  - In `Linux Kernel Tools`, select `perf`
- `Target packages`
  - `Debugging, profiling and benchmark`
    - Select `ltrace`
    - Select `strace`

Then rebuild and update your NFS root filesystem.

Using strace

Let's run the program through the `strace` command to find out why this happens.

You should see that it's trying to access files that don't exist. Once you've found what's wrong, fix the code (or ask your instructor for help if needed), then rebuild the program and run it again:

```
# /root/nunchuk-mpd-client
ERROR: didn't manage to find the Nunchuk device in /dev/input. Is the Nunchuk driver loaded?
```

Ouch, same problem again!

You can run the program again through `strace`, and check that the right paths are now accessed, but the cause of the issue won't be easy to find.

Using ltrace

Let's run the program through `ltrace` now. We will be able to see the shared library calls.

Take your time to study the `ltrace` output. That's interesting information! Back to our issue, the last lines of output should make the issue pretty obvious.

Fix the bug in the code, recompile the program, copy it to the target, strip it and start it again.

You should now be able to use the new client, driving the server through the following Nunchuk inputs:

- Joystick up: volume up 5%
- Joystick down: volume down 5%
- Joystick left: previous song
- Joystick right: next song

 © 2004-2026 Bootlin, CC BY-SA license

- Z (big) button: pause / play
- C (small) button: quit client

Have fun with the new client. You'll just realize that quitting causes the program to crash with a segmentation fault. Let's debug this too.

Using gdbserver from the command line

We are going to use `gdbserver` to understand why the program segfaults.

Compile `nunchuk-mpd-client.c` again with the `-g` (`g` means *gdb*) option to include debugging symbols. This time, just keep it on your workstation, as you already have the version without debugging symbols on your target.

Then, on the target side, run the program under `gdbserver`. `gdbserver` will listen on a TCP port for a connection from `gdb` on the host, and will control the execution of `nunchuk-mpd-client` according to the `gdb` commands:

```
=> gdbserver localhost:2345 /root/nunchuk-mpd-client
```

On the host side, run `arm-linux-gdb` (also found in your toolchain):

```
$ arm-linux-gdb nunchuk-mpd-client
```

`gdb` starts and loads the debugging information from the `nunchuk-mpd-client` binary (in the `appdev` directory) which has been compiled with `-g`.

Then, we need to tell where to find our libraries, since they are not present in the default `/lib` and `/usr/lib` directories on your workstation. This is done by setting the `gdb sysroot` variable (on one line):

```
(gdb) set sysroot /home/<user>/embedded-linux-bbb-labs/integration/\
   buildroot/output/staging
```

Of course, replace `<user>` by your actual user name.

And tell `gdb` to connect to the remote system:

```
(gdb) target remote <target-ip-address>:2345
```

Then, use `gdb` as usual to set breakpoints, look at the source code, run the application step by step, etc.

In our case, we'll just start the program, press the `C` button to quit to cause the the segmentation fault:

```
(gdb) continue
```

After the segmentation fault, you can ask for a backtrace to see where this happened:

```
(gdb) backtrace
```

This will tell you that the segmentation fault occurred in a function of the `libmpdclient`, called by our program. You will also get the number of the line in the program which caused this. This should help you to find the bug in our application.

Once you found it, don't fix it yet. We are going to make further experiments around this segmentation fault.

Post mortem analysis

By default `systemd` disables generating `core` files, so we need to re-enable the generation of `core` files by running on the target:

- `echo core.%p > /proc/sys/kernel/core_pattern`, which will replace the `|/bin/false` that was set by *systemd* during boot
- `ulimit -c unlimited` to make sure no size limit is imposed on core files

Run `nunchuk-mpd-client` again, and exit by pressing `C` on the joystick, it should generate the crash, which in turn should cause a `core.<pid>` file to be generated.

Once you have such a file, inspect it with `arm-linux-gdb` on the host as explained in the lectures.

Don't be surprised, the below warnings are expected:

```
warning: Can't open file /root/nunchuk-mpd-client during file-backed mapping note processing
warning: Can't open file /usr/lib/libc.so.6 during file-backed mapping note processing
warning: Can't open file /usr/lib/libmpdclient.so.2.22 during file-backed mapping note processing
warning: Can't open file /usr/lib/ld-linux-armhf.so.3 during file-backed mapping note processing
warning: core file may not match specified executable file.
```

In the `gdb` shell, set the `sysroot` setting as previously, and then generate a backtrace to see where the program crashed. You can even see the value of all variables in the different function contexts of your program:

```
(gdb) bt full
```

This way, you can have a lot of information about the crash without running the program through the debugger.

Editing and remote compiling with VS Code

Installing software

We are going to use Visual Studio Code to do the remote debugging again, and eventually fix and recompile our program.

The first thing to do is install VS Code. This package is only available as a *snap package*:

```
$ sudo snap install --classic code
```

Preparing the target for debugging with VSCode

We will use Visual Studio Code to modify and recompile our client program, and also to update and run the binary on the target. Of course, we will use a simple solution, as we won't be able to spend too much time learning about all the possibilities offered by VS Code.

For our purpose, a good solution is SSH, which allows to copy files (through the `scp` command) and to run remote commands. We already included the *Dropbear* SSH server in our root filesystem.

We just need to implement password-less SSH access, to keep things simple:

- If you don't have an SSH key yet (look at `~/.ssh/`), generate a password-less one with the `ssh-keygen` command. By default, this creates two files in `~/.ssh/`: `id_ed25519` (private key) and `id_ed25519.pub` (public key).
- Then create a `/root/.ssh/authorized_keys` file **on the target** with the line in `id_ed25519.pub`.
- Then, fix permissions on the target, as Dropbear is quite strict about them:

```
# chmod -R go-rwx /root
# chown -R root:root /root
```

Then, you can test that SSH works without a password:

 © 2004-2026 Bootlin, CC BY-SA license

```
ssh root@192.168.0.100
```

If you face trouble, you can check the Dropbear logs on the target:

```
journalctl -fu dropbear
```

Additionally, before being able to debug the our application on the target, we need to make sure that a gdbserver instance is running and can be accessed by VSCode. We can add a small systemd service to handle this. The minimal code to enable such service looks like this:

```
[Unit]
Description=GDB server for application debugging
After=network.target
Wants=network.target

[Service]
Type=exec
ExecStart=/usr/bin/gdbserver --multi :3333
Restart=always

[Install]
WantedBy=multi-user.target
```

Copy this snippet and paste it in a new `gdbserver.service` file onto the target filesystem, in `/usr/lib/systemd/system/`. Then enable the service and make it start automatically during each boot:

```
systemctl daemon-reload && systemctl enable --now gdbserver
```

Compiling and debugging the program from VS Code

The `appdev` directory already contains a `.vscode` directory with ready made settings for code editing and for compiling and debugging our application. Here are these files:

- `.vscode/c_cpp_properties.json`: settings for the code editor.
- `.vscode/tasks.json`: definition of some standard project management tasks, like ”build”, ”clean” and ”deploy” tasks
- `.vscode/launch.json`: these are the settings for remote debugging.
- `.vscode/extensions.json`: some needed extensions pinned as ”recommended” so that you can find those easily in the plugins menu
- `.env`: some user-defined to let all the files above know how to access the target.

First, start VS Code:

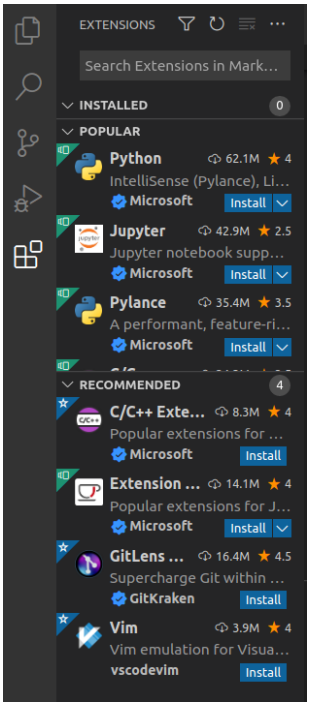
```
$ code
```

Use `File` → `Open Folder` to open the `appdev` directory.

The first thing to do is to make sure that some needed extensions are installed. To do so, switch to the plugins section in the vertical tab, then search and install the following extensions:

- `C/C++` extension from Microsoft (`ms-vscode.cpptools`)
- `Tasks Shell Input` extension (`augustocdias.tasks-shell-input`)

Those extensions should appear on top of the ”Recommended” section.

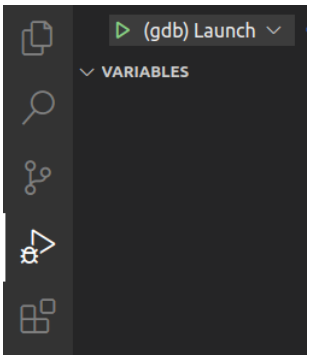


Then open the `.env` file and if needed, update the `TARGET_IP` variable to make it match the IP address of the target.

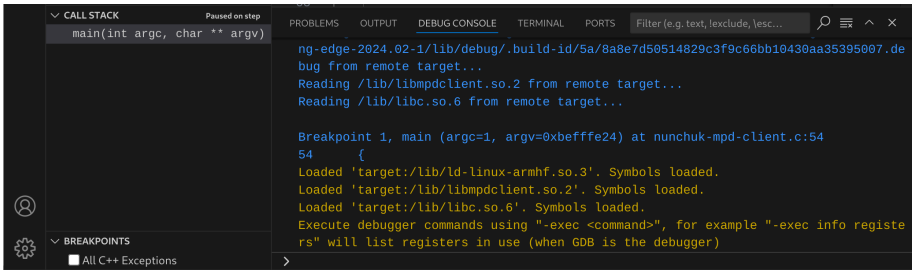
You are now ready to build and debug your application. Start by clicking on the `nunchuk-mpd-client.c` file in the left column to open it in VS Code. Build it thanks to the tasks configured in VSCode: bring the Command Palette by typing `Ctrl+Shift+P`, then type "Run Build Task", then Enter.

Note: you can also use the `Ctrl+Shift+B` *shortcut to execute the default build action*

You can start debugging the program by clicking on the `Run and Debug` tab, and then on the green "Play" button on top:



In the debug console, you should see that debugging has started:



 © 2004-2026 Bootlin, CC BY-SA license

Note: VSCode will automatically rebuild and redeploy the application each time you start a debugging session.

Then, start using the Nunchuk to control playback, and when you try to quit with the `C` button, VS Code should now see the segmentation fault:



You can then look at variables, the call stack, browse the code...

To stop debugging, you should use `Run` → `Stop Debugging`.

By studying the the code, you should eventually find that what's causing the segmentation fault is the call to `free()` in the test for the `C` button. Remove this line, save the file through the `File` menu (otherwise nothing will change), and then compile and run the application again. This time, there should be no more segmentation fault when you hit the `C` button.

If you are ahead of time, don't hesitate to spend more time with VS Code, for example to add breakpoints and execute the program step by step.

Profiling the application with perf

Let's make a quick attempt at profiling our application with the `perf` command:

```
perf record /root/nunchuk-mpd-client
```

Use your application and leave it when you are done.

This stores profiling data in a `perf.data` file. One way to extract information from it is to run the below command in the same directory (the one containing `perf.data`):

```
perf report
```

See the time spent in various kernel (`[k]`) and userspace (`[.]`) functions. The details of the kernel functions is not visible, but additional symbol information can be added by building the kernel with the `CONFIG_KALLSYMS_ALL` option enabled.

Now, let's profile the whole system. First, make sure that the system is currently playing audio. Then SSH to your board and run `perf top` (working better through SSH) to see live information about kernel and userspace functions consuming most CPU time.

This is interactive, but hard to analyze. You can also run `perf record` for about 30 seconds, followed by `perf report` to have a useful summary of system wide activity for a substantial amount of time.

This was a very brief start at practising with `perf`, which offers many more possibilities than we could see here.

© 2004-2026 Bootlin, CC BY-SA license

What to remember

During this lab, we learned that...

- It's easy to study the behavior of programs and diagnose issues without even having the source code, thanks to `strace`, `ltrace` and `perf`.
- You can use `perf` as a system wide profiler too.
- You can leave a small `gdbserver` program (about 400 KB) on your target that allows to debug target applications, using a standard `gdb` debugger on the development host, or a graphical IDE such as VS Code.
- It is fine to strip applications and binaries on the target machine, as long as the programs and libraries with debugging symbols are available on the development host.
- Thanks to `core` dumps, you can know where a program crashed, without having to reproduce the issue by running the program through the debugger.

Going further: packaging your application with Meson

Now that our application is ready, the next thing to do is to properly integrate it into our root filesystem. This is a nice opportunity to see how to do this with *Meson* and leverage Buildroot's infrastructure to cross-compile *Meson* based packages.

Still in the main `appdev` directory, create a `nunchuk-mpd-client-1.0` directory and copy the `nunchuk-mpd-client.c` file to it.

In this new directory, all you have to do is create a very simple `meson.build` file:

```
project('nunchuk-mpd-client', 'c', version: '1.0')
libmpdclient_dep = dependency('libmpdclient', version: '>= 2.16')
executable('nunchuk-mpd-client', 'nunchuk-mpd-client.c',
           dependencies: libmpdclient_dep, install: true)
```

Note that `install: true` is necessary to get the executable installed by `ninja install`.

Now, the next thing is to add a new package to the Buildroot source tree:

- Create a `nunchuk-mpd-client` directory under `package`.
- In this directory, create a `Config.in` file. You can reuse the one from the `mpd-mpc` package (the *mpc* client) which also depends on *libmpdclient*.
- Modify `package/Config.in` to source this new file in the `Audio and video applications` submenu.
- Last but not least, create the `nunchuk-mpd-client.mk` file with the following contents:

```
################################################################################
#
# nunchuk-mpd-client
#
################################################################################

NUNCHUK_MPD_CLIENT_VERSION = 1.0
NUNCHUK_MPD_CLIENT_SITE = $(HOME)/embedded-linux-bbb-labs/appdev/nunchuk-mpd-client-1.0
NUNCHUK_MPD_CLIENT_SITE_METHOD = local
NUNCHUK_MPD_CLIENT_DEPENDENCIES = host-pkgconf libmpdclient

$(eval $(meson-package))
```

All you have to do now is to enable the `nunchuk-mpd-client` package in Buildroot's configuration, run `make`, update the root filesystem and check on the target that `/usr/bin/nunchuk-mpd-client` exists and runs fine.

 © 2004-2026 Bootlin, CC BY-SA license

All this was pretty straightforward, wasn't it? *Meson* rocks!

Congratulations, you've reached the end of all our labs. Try to look back, and see how much experience you've gained in these last days.

© 2004-2026 Bootlin, CC BY-SA license